

Sicurezza delle Reti

Riassunto delle parti viste a lezione del libro "Computer Security – Principles and practice"

INTRODUZIONE	3
1 OVERVIEW	3
1.1 COMPUTER SECURITY CONCEPTS – CONCETTI E TERMINOLOGIA DELLA SICUREZZA INFORMATICA	3
1.1.1 A Definition of Computer Security – Definizione di Computer Security	3
1.1.2 Examples – Esempi	4
1.1.3 The Challenges of Computer Security – Le Sfide della Sicurezza Informatica	4
1.1.4 A Model for Computer Security – Terminologia	5
1.2 THREATS, ATTACKS, AND ASSETS – MINACCE, ATTACCHI E RISORSE	6
1.2.1 Threats and Attacks – Minacce e Attacchi	6
1.2.2 Threats and Assets – Minacce e Risorse	6
1.4 FUNDAMENTAL SECURITY DESIGN PRINCIPLES – PRINCIPI FONDAMENTALI PER IL DESIGN	8
1.5 ATTACK SURFACES AND ATTACK TREE – SUPERFICI E ALBERI DI ATTACCO	9
1.5.1 Attack Surfaces – Superfici di Attacco	9
1.5.2 Attack Trees – Alberi di Attacco	9
1.6 COMPUTER SECURITY STRATEGY – STRATEGIE DELLA SICUREZZA INFORMATICA	9
1.6.1 Security policy – Politiche di sicurezza	9
1.6.2 Security Implementation - Implementazioni della sicurezza	10
1.6.3 Assurance and Evaluation – Garanzia e Valutazione	10
2 CRYPTOGRAPHIC TOOLS – CRITTOGRAFIA	10
2.1 CONFIDENTIALITY WITH SYMMETRIC ENCRYPTION – CONFIDENZIALITÀ CON CRITTOGRAFIA A CHIAVE SIMMETRICA	10
2.1.1 Symmetric Encryption – Crittografia Simmetrica	10
2.1.2 Symmetric Block Encryption Algorithm – Algoritmi di crittografia chiave simmetrica a blocchi	11
2.1.2.1 Data Encryption Standard (DES)	11
2.1.2.2 Triple DES (3DES)	11
2.1.2.3 Advanced Encryption Standard (AES)	11
2.1.2.4 Practical Security Issues – Problemi di sicurezza degli algoritmi a blocchi	11
2.2 MESSAGE AUTHENTICATION AND HASH FUNCTION – MESSAGE AUTHENTICATION E FUNZIONI HASH	12
2.2.1 Authentication using Symmetric Encryption – Autenticazione con crittografia a chiave simmetrica	12
2.2.2 Message Authentication without Message Encryption – Autenticazione senza crittografia	12
2.2.3 Message Authentication Code	12
2.2.4 One-Way Hash Function – Funzioni Hash	13
2.2.5 Secure Hash Function – Funzioni Hash sicure	13
2.2.5.1 Security of Hash Functions – Sicurezza delle Funzioni Hash	14
2.2.5.2 Secure Hash Function Algorithms	14
2.2.6 Other applications of Hash Function – Altre applicazioni delle Funzioni Hash	14
2.3 PUBLIC-KEY ENCRYPTION - CRITTOGRAFIA A CHIAVE PUBBLICA	14
2.3.1 Public-Key Encryption Structure – Struttura della Crittografia a Chiave Pubblica	14
2.3.2 Applications for Public-Key Cryptosystems - Applicazioni della Crittografia a Chiave Pubblica	15
2.3.3 Requirements for Public-Key Cryptography – Requisiti per la Crittografia a Chiave Pubblica	15
2.3.4 Asymmetric Encryption Algorithm – Algoritmi di Crittografia a Chiave Asimmetrica	15
2.3.4.1 RSA	15
2.4 DIGITAL SIGNATURES AND KEY MANAGEMENT – FIRME DIGITALI E GESTIONE DELLE CHIAVI	16
2.4.1 Digital Signatures – Firme Digitali	16
2.4.2 Public-Key Certificates – I Certificati per le Chiavi Pubbliche	16
2.4.3 Symmetric Key Exchange Using Public-Key Encryption – Condivisione di Chiavi Simmetriche	16
2.4.4 Digital Envelopes	16
2.5 RANDOM AND PSEUDORANDOM NUMBER – NUMERI CASUALI E PSEUDOCASUALI	17
2.5.1 The Use of Random Numbers - L'uso di Numeri Casuali	17
2.5.1.1 Randomness - Casualità	17
2.5.1.2 Unpredictability – Casualità	17
2.5.2 Random versus Pseudorandom	17
2.6 PRATICAL APPLICATION: ENCRYPTION OF STORED DATA – APPLICAZIONE PRATICA: MEMORIZZAZIONE DI DATI CRIPTATI	17
3 USER AUTHENTICATION	17
3.1 ELECTRONIC USER AUTHENTICATION PRINCIPLES – I PRINCIPI DELLA USER AUTHENTICATION	18
3.1.1 A Model for Electronic User Authentication – Un modello per la User Authentication	18

3.1.2	Means of Authentication – Metodi di Autenticazione	18
3.2	PASSWORD-BASED AUTHENTICATION – AUTENTICAZIONE CON PASSWORD.....	18
3.2.1	The Vulnerability of Password – Vulnerabilità delle Password	18
3.2.2	The Use of Hashed Passwords – Hash delle Password.....	19
3.2.2.1	UNIX Implementations – Implementazione UNIX.....	19
3.2.3	Password Cracking of User-Chosen Passwords – Cracking di Password	20
3.2.3.1	Traditional Approach – Approcci classici	20
3.2.3.2	Modern Approach – Approccio attuale	20
3.2.4	Password File Access Control – Controllo degli accessi al file delle password	20
3.2.5	Password Selection Strategy – Strategie di selezione della password	20
3.3	TOKEN-BASED AUTHENTICATION – AUTENTICAZIONE BASATA SU TOKEN	21
3.3.1	Memory Cards	21
3.3.2	Smart Cards	21
3.5	REMOTE USER AUTHENTICATION.....	21
3.5.1	Password Protocol	21
3.6	SECURITY ISSUES FOR USER AUTHENTICATION – PROBLEMI DI SICUREZZA NELLA USER AUTHENTICATION	22
4	ACCESS CONTROL – CONTROLLO DEGLI ACCESSI.....	22
4.1	ACCESS CONTROL PRINCIPLES – PRINCIPI DEL CONTROLLO DEGLI ACCESSI	22
4.1.1	Access Control Context - ??.....	23
4.1.2	Access Control Policies – Politiche per il Controllo degli Accessi	23
4.2	SUBJECTS, OBJECTS AND ACCESS RIGHTS – SOGGETTI, OGGETTI E DIRITTI DI ACCESSO	23
4.3	DISCRETIONARY ACCESS CONTROL	24
4.4	EXAMPLE: UNIX FILE ACCESS CONTROL – ESEMPIO: ACCESS CONTROL IN UNIX	24
4.4.1	Traditional UNIX File Access Control	24
4.4.2	Access Control List in UNIX	25
4.5	ROLE-BASED ACCESS CONTROL	25
4.5.1	RBAC Reference Models – Modelli di Riferimento per RBAC.....	25
4.5.1.1	Base Model – RBAC ₀ – Modello base	26
4.5.1.2	Role Hierarchies – RBAC ₁ – Gerarchie	26
4.5.1.3	Constraint – RBAC ₂ – Vincoli	26
5	DATABASE AND CLOUD SECURITY	26
5.4	DATABASE ACCESS CONTROL	26
5.4.1	A Typical SQLi Attack – Un tipico attacco SQL Injection.....	26
5.4.2	The Injection Technique – Tecnica per l'iniezione di codice	27
5.4.3	SQLi Attack Avenues and Types – Modalità di attacco SQLi	27
5.4.4	SQLi Countermeasures.....	27
5.8	CLOUD COMPUTING	28
5.8.1	Cloud Computing Reference Architecture – Architettura di riferimento per il Cloud	29
5.9	CLOUD SECURITY RISKS AND COUNTERMEASURES – RISCHI E CONTROMISURE PER SERVIZI CLOUD	29
5.10	DATA PROTECTION IN THE CLOUD – PROTEZIONE DEI DATI NEL CLOUD	30
5.11	CLOUD SECURITY AS A SERVICE – SECAAS.....	30
9	FIREWALLS AND INTRUSION PREVENTION SYSTEMS – FIREWALL E INTRUSION PREVENTION	31
9.1	THE NEED FOR FIREWALLS – LO SCOPO DEI FIREWALL	31
9.2	FIREWALL CHARACTERISTICS AND ACCESS POLICY – CARATTERISTICHE DEI FIREWALL	31
9.3	TYPES OF FIREWALLS – TIPI DI FIREWALL	32
9.3.1	Packet Filtering Firewall	32
9.3.2	Stateful Inspection Firewalls.....	33
9.3.3	Application-Level Gateway.....	33
9.3.4	Circuit-Level Gateway.....	33
9.4	FIREWALL BASING – POSIZIONAMENTO DEI FIREWALL SUGLI HOST	33
9.4.1	Bastion Host	34
9.4.2	Host-Based Firewalls	34
9.4.3	Personal Firewall	34
9.5	FIREWALL LOCATION AND CONFIGURATION – POSIZIONAMENTO DEI FIREWALL IN RETE E CONFIGURAZIONE	34
9.5.1	DMZ Networks – Demilitarized Networks.....	34
9.5.2	Virtual Private Network	34
9.5.3	Distributed Firewalls – Firewall distribuiti	35
9.5.4	Summary of Firewall Locations and Topology – Riassunto sulle tecniche di posizionamento dei Firewall	35

Introduzione

Non esistono sistemi sicuri

Il **grado di sicurezza** del sistema è dato dal **tempo** necessario per violarlo, dall'**investimento** necessario e dalla **probabilità di successo**.

Principio 1) Il software non è perfetto e potrebbero esistere **errori di progettazione** nei protocolli e nelle procedure usate. **Non esiste** un sistema **inviolabile**.

Corollario) La **crescita di complessità di un sistema** porta alla creazione di **sistemi insicuri**. Rende infatti più probabili errori di progettazione, di implementazioni, di utilizzo, con conseguenti falle di sicurezza e difficoltà nel comprendere il comportamento del sistema e le interazioni tra le varie componenti. Per questo motivo è importante cercare di mantenere il sistema e i meccanismi di sicurezza quanto più possibile semplici.

Principio 2) Il **software** non è l'unica componente da mantenere sicura, ma vi sono anche l'**hardware** (parte molto difficile da proteggere) e la **componente umana** (spesso anello debole della catena). Fanno parte dei sistemi di sicurezza la progettazione sicura, le politiche di accesso, la protezione fisica e la formazione degli utenti.

Non è un buon sistema di sicurezza la *security through obscurity*: quando si progetta un sistema è necessario pensare che ciò che garantisce protezione non è la sicurezza dell'algoritmo utilizzato (che spesso è di pubblico dominio), ma la sicurezza della chiave.

Principio 3) La sicurezza non è un'azione, ma un **processo** che non termina mai. Nessun sistema può essere considerato sicuro per sempre. Esso deve quindi essere mantenuto aggiornato. Gli utenti che lo utilizzano devono inoltre essere continuamente educati.

1 Overview

Nel capitolo verrà spiegata la definizione di *sicurezza informatica*. In seguito verranno elencate le categorie di *risorse* che utenti e amministratori di sistema sono interessati a preservare e proteggere, analizzando le possibili *minacce* che possono colpirle e le *contromisure* che possono essere prese.

1.1 Computer Security Concepts – Concetti e terminologia della sicurezza informatica

1.1.1 A Definition of Computer Security – Definizione di Computer Security

Computer security: protezione di un sistema automatico o di un sistema informatico attuata per perseguire l'obiettivo di preservare *integrità*, *disponibilità* e *confidenzialità* delle risorse del sistema (hardware, software, informazioni/dati, telecomunicazioni).

- **Confidenzialità (Confidentiality):**
 - **Confidenzialità dei dati:** le informazioni private vengono mantenute private e risultano non disponibili a individui non autorizzati.
 - **Privacy:** l'utente è in grado di controllare la diffusione e la memorizzazione delle sue informazioni (Quali? A chi? Da chi?)
- **Integrità (Integrity):**
 - **Integrità dei dati:** informazioni e programmi vengono modificati solo in maniera autorizzata (es. durante un aggiornamento). Sono incluse la non-repudiation (non è possibile dire "Non ho mandato io questo messaggio") e l'autenticità (il messaggio non è stato modificato durante il tragitto).
 - **Integrità del sistema:** il sistema esegue solo le operazioni per cui è stato progettato, senza deliberati o inavvertiti effetti malevoli.
- **Disponibilità (Availability):** il sistema funziona correttamente e il servizio è disponibile agli utenti autorizzati, che possono usare le risorse messe a disposizione.

Alle tre caratteristiche elencate nella definizione vengono aggiunti due concetti aggiuntivi:

- **Autenticità (Authenticity):** la proprietà di un dato di essere verificato e di potervi fare affidamento, garantendo l'affidabilità e validità del canale di trasmissione, del messaggio e dell'origine.

- **Tracciabilità (Accountability):** possibilità di associare un'azione a una certa entità in maniera inequivocabile. Ciò supporta la non-repudiation, la possibilità di rilevare intrusioni (e prevenirle fungendo da deterrente), la possibilità di intraprendere azioni legali a seguito, ...

1.1.2 Examples – Esempi

L'ente di standardizzazione definisce 3 livelli di gravità di una breccia nella sicurezza:

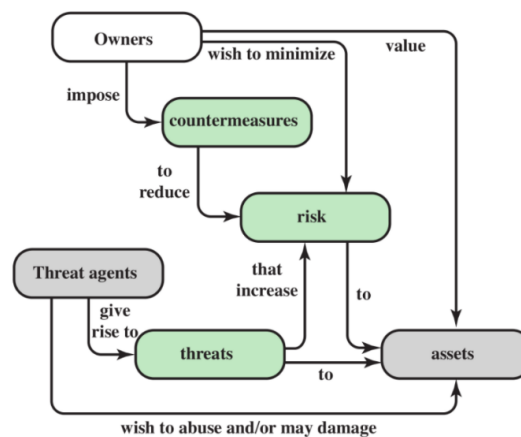
- **Basso:** le ripercussioni sulle risorse, sulle funzioni aziendali e sugli individui dell'organizzazione sono limitate. Potrebbero coinvolgere:
 - Limitata degradazione delle performance dell'impresa
 - Danni minori alle risorse aziendali
 - Danni economici di lieve entità
 - Danni leggeri al personale
- **Medio:** le ripercussioni sulle risorse, sulle funzioni aziendali e sugli individui dell'organizzazione sono gravi. Potrebbero coinvolgere:
 - Significativa degradazione delle performance dell'impresa
 - Danni significativi alle risorse aziendali
 - Danni economici significativi
 - Danni al personale che non includono morte o minacce alla vita
- **Alto:** le ripercussioni sulle risorse, sulle funzioni aziendali e sugli individui dell'organizzazione sono molto gravi o catastrofiche. Potrebbero coinvolgere
 - Grave degradazione delle performance dell'impresa
 - Danni gravi alle risorse aziendali
 - Danni economici particolarmente importanti
 - Morte o minacce alla vita del personale

1.1.3 The Challenges of Computer Security – Le Sfide della Sicurezza Informatica

La sicurezza informatica è una disciplina molto complessa, infatti:

- La maggior parte dei requisiti sono descritti da parole semplici ed auto-esplicative, ma i *meccanismi* che permettono di raggiungerli sono *complessi* da raggiungere e richiedono molto *ragionamento*.
- Bisogna sempre considerare *tutti i possibili tipi di attacco* a un meccanismo o algoritmo di sicurezza: gli attacchi che hanno successo spesso sono pianificati osservando il sistema da una luce completamente diversa.
- A causa di questo, le procedure da mettere in atto sono spesso *controintuitive*, *complesse* e *non ovvie*. Solo quando tutti gli aspetti della minaccia vengono considerati il meccanismo può essere funzionante.
- Non è importante solo decidere quali sistemi utilizzare, ma anche *dove* utilizzarli (sia fisicamente, nello spazio, che logicamente).
- Oltre all'algoritmo è necessario considerare che esso prevede anche la conservazione di *informazioni segrete* (chiavi, ...). Sono necessari protocolli per definire come creare, distribuire e proteggere tali informazioni.
- La lotta è *fortemente asimmetrica*: l'amministratore di sistema deve ricercare e proteggere tutti i possibili buchi, mentre l'attaccante si limita a trovare una sola debolezza per poter danneggiare il sistema.
- Utenti e anche amministratori di sistema percepiscono poco il problema della sicurezza fino a che non si verifica un attacco.
- Per garantire la sicurezza del sistema è necessario *monitorare regolarmente* il sistema.
- La sicurezza è spesso un pensiero *secondario* in fase di design e deve essere integrata solo in un secondo momento.
- Molti utenti e amministratori di sistema vedono la sicurezza come una *limitazione* all'efficienza e alla user-friendliness del sistema.

1.1.4 A Model for Computer Security – Terminologia



- **Risorse di sistema (System resource, Assets):** risorse che il proprietario del sistema vuole proteggere. Comprendono:
 - **Hardware:** elaboratori, unità di calcolo, unità di memorizzazione, dispositivi per la comunicazione in rete.
 - **Software:** sistemi operativi, applicazioni.
 - **Dati:** file, database, dati relativi alla sicurezza (file delle password, chiavi, ...).
 - **Reti e canali di comunicazione:** LAN, WAN, bridge, router, cavi, ...
- **Avversario, Agente di minaccia, Attaccante (Adversary, Threat agent):** entità che attacca o minaccia un sistema.
- **Attacco (Attack):** attacco alla sicurezza di un sistema che deriva da una *minaccia intelligente*, un *tentativo deliberato* di evadere ai sistemi e alle politiche di sicurezza di un sistema. Gli attacchi possono essere divisi in due modi:
 - **Attivo vs Passivo:** un attacco viene definito attivo quando va ad influenzare le capacità del sistema, mentre viene definito passivo quando l'attaccante semplicemente raccoglie o analizza le informazioni del sistema senza danneggiarle.
 - **Inside vs Outside:** un attacco può essere condotto dall'interno (l'utente è autorizzato ad utilizzare il sistema ma non nel modo in cui sta cercando di farlo) oppure dall'esterno (l'utente non è autorizzato ad utilizzare il sistema).
- **Contromisure (Contermeasures):** azioni, dispositivi, procedure e tecniche che riducono una minaccia, una vulnerabilità o un attacco prevenendo o minimizzando i danni che esso causa oppure scoprendolo e riportandolo in modo da poter prendere azioni correttive.
- **Rischio (Risk):** possibilità di perdita o danno espressa come probabilità che una minaccia sfrutti una particolare vulnerabilità con un certo risultato dannoso.
- **Politica di Sicurezza (Security Policy):** insieme di regole che determinano l'organizzazione del sistema e permettono il funzionamento dei servizi di sicurezza proteggendo le risorse del sistema.
- **Minaccia (Threat):** potenziale violazione della sicurezza che esiste quando c'è una circostanza, un'azione o un evento che potrebbe bucare la sicurezza. Qualcosa che può sfruttare una vulnerabilità.
- **Vulnerabilità (Vulnerability):** debolezza nell'implementazione, nel design o nella conduzione del sistema che potrebbe essere sfruttata per violare le politiche di sicurezza. Può essere di 3 tipi:
 - **Corruzione:** il sistema funziona in maniera non adeguata (esegue operazioni non richieste o risponde in maniera errata). Cfr. integrità.
 - **Perdita (Leak):** le informazioni nascoste vengono rese disponibili a qualcuno che non dovrebbe avere l'accesso. Cfr. confidenzialità.
 - **Non disponibilità:** il servizio diventa non utilizzabile o estremamente lento. Cfr. disponibilità.

1.2 Threats, Attacks, and Assets – Minacce, Attacchi e Risorse

Verranno descritti i vari tipi di minaccia esistenti e quali tipologie di risorse esse danneggiano.

1.2.1 Threats and Attacks – Minacce e Attacchi

- **Diffusioni non autorizzate (Unhauthorized disclosure):** minaccia alla *confidenzialità*. Un'entità ottiene l'accesso a un dato che non è autorizzato a possedere. Avviene in diversi modi:
 - **Esposizione (Exposure):** i dati vengono concessi da un insider ad un'entità non autorizzata (outsider). Ciò si può verificare a causa di errori umani, hardware o software.
 - **Intercettazione (Interception):** un'entità non autorizzata accede ai dati che stanno viaggiando tra mittente e destinatario. È possibile che un attacco di questo tipo di verifichi in LAN (dove tutti possono ricevere pacchetti nonostante non siano i destinatari prescelti) o su Internet (accedendo, per esempio, al traffico email o allo scambio di altri dati).
 - **Inferenza (Inference):** un'entità non autorizzata riesce ad accedere indirettamente ai dati (non necessariamente il contenuto della comunicazione, ma anche altri dati rilevanti) analizzando le caratteristiche della comunicazione. Un esempio è la traffic analysis, in cui l'attaccante ricava informazioni osservando il pattern del traffico in rete, oppure la possibilità di rilevare dati complessi facendo particolari interrogazioni a un database su cui si ha accesso limitato e incrociando i risultati.
 - **Intrusione (Intrusion):** un'entità non autorizzata ottiene l'accesso a dati sensibili bypassando le misure di sicurezza di un sistema.
- **Inganno (Deception):** minaccia all'*integrità dei dati*. Un'entità autorizzata ad accedere al sistema riceve false informazioni credendo però che siano vere. Avviene in diversi modi:
 - **Mascheramento (Masquerade):** un'entità non autorizzata ottiene l'accesso al sistema o esegue operazioni facendosi credere un utente del sistema. Avviene quando si riescono a scoprire le credenziali di accesso oppure tramite Trojan horse (software che sembrano avere certe funzionalità mentre in realtà ottengono l'accesso non autorizzato al sistema).
 - **Falsificazione (Falsification):** un'entità autorizzata riceve dati falsificati. Si ha quando un'entità non autorizzata modifica o aggiunge dati falsi a un database o un file.
 - **Ripudio (Repudiation):** un'entità inganna le altre affermando di non aver compiuto una certa operazione oppure di possedere o non possedere certi dati.
- **Interruzione (Disruption):** minaccia alla *disponibilità* o all'*integrità del sistema*. Un evento o una circostanza interrompono o impediscono l'operatività del sistema.
 - **Inabilitazione (Incapacitation):** attacco alla *disponibilità*. Un sistema viene messo fuori servizio disabilitando uno o più componenti. Può avvenire distruggendo o danneggiando sia parti software che parti hardware.
 - **Corruzione (Corruption):** attacco all'*integrità del sistema*. Un software malevolo o l'intervento di un'entità non autorizzata fa sì che il sistema funzioni in una maniera non prevista (es. inserendo backdoor).
 - **Ostruzione (Obstruction):** attacco alla *disponibilità*. Viene impedito l'accesso al servizio per gli utenti autorizzati sovraccaricando il sistema oppure interferendo con le comunicazioni.
- **Usurpazione (Usurpation):** attacco all'*integrità del sistema*. Un'autorità non autorizzata prende il controllo dei servizi o delle funzioni del sistema.
 - **Appropriazione indebita (Misappropriation):** un'entità non autorizzata assume il controllo fisico di una risorsa del sistema. Si ha per esempio quando viene sfruttata la potenza di calcolo di macchine infettate per eseguire attacchi DDoS.
 - **Abuso (Misuse):** un componente del sistema si comporta in modo non coerente con la sicurezza del sistema, danneggiando o rendendo impossibile il controllo della sicurezza.

1.2.2 Threats and Assets – Minacce e Risorse

Un sistema è formato da diversi tipi di risorsa: hardware, software, dati e linee di comunicazione. Tutti e quattro i tipi di risorsa devono essere adeguatamente protetti per garantirne *integrità*, *confidenzialità* e *disponibilità*.

- **Hardware:** l'hardware è la parte più vulnerabile ad attacchi alla *disponibilità*, in quanto può essere accidentalmente o deliberatamente danneggiato oppure rubato.

- **Disponibilità:** furto o disabilitazione di risorse hardware, impedendo la fornitura del servizio.
- **Confidenzialità:** furto di supporti di memorizzazione non criptati.
- **Integrità:** aggiunta di componenti hardware in grado di modificarne il comportamento (es. per acquisire informazioni).
- **Software:** comprende OS e applicazioni. È vulnerabile a diversi tipi di attacchi ma non è sempre facile da proteggere.
 - **Disponibilità:** cancellazione di parti del programma. Una contromisura da prendere è fare backup periodici.
 - **Confidenzialità:** clonazione non autorizzata del software. Non esistono contromisure definitive.
 - **Integrità:** modifica o aggiunta di funzioni per ottenere comportamenti non previsti.
- **Dati:** comprendono tutti i file e le altre forme di memorizzazione controllate dagli utenti.
 - **Disponibilità:** i file possono essere cancellati o comunque resi inaccessibili agli utenti.
 - **Confidenzialità:** lettura non autorizzata di informazioni o analisi statistiche su dati disponibili per scoprire informazioni nascoste.
 - **Integrità:** modifica o aggiunta di nuovi file.
- **Linee di comunicazione e reti:** parte dei sistemi più attaccata.
 - **Disponibilità:** distruzione o cancellazione dei messaggi oppure messa fuori servizio delle linee.
 - **Confidenzialità:** lettura dei messaggi oppure analisi sul pattern di comunicazione.
 - **Integrità:** modifica, ritardo, riordinamento, duplicazione o creazione di falsi messaggi.

Esistono due tipi principali di attacco:

- **Attacchi passivi:** eseguiti per acquisire informazioni sul sistema ma senza danneggiare direttamente le risorse di sistema. Attacchi di questo tipo sono l'eavesdropping (origliare) o monitoraggio delle trasmissioni, che ha come scopo raccogliere informazioni per poterle riutilizzare in un secondo momento.

Gli attacchi passivi sono particolarmente complessi da riconoscere in quanto non coinvolgono l'alterazione dei dati. L'unico modo di prevenirli è usare la crittografia.

Esistono due tipi di attacco:

- **Lettura dei messaggi:** i messaggi, che potrebbero contenere informazioni riservate, vengono direttamente letti dall'attaccante. La crittografia è una contromisura efficiente.
- **Analisi del traffico:** tipo di attacco più subdolo in cui vengono ottenute informazioni interessanti sulla natura delle comunicazioni senza leggerne direttamente il contenuto ma solo analizzando caratteristiche esteriori (es. dimensione dei pacchetti, frequenza della trasmissione, ...). La crittografia non è efficiente per prevenire attacchi di questo tipo.
- **Attacchi attivi:** eseguiti con la specifica intenzione di alterare le risorse di sistema, danneggiandone il corretto funzionamento.

Gli attacchi attivi sono spesso più semplici da riconoscere, ma difficili da prevenire. Per questo è importante essere in grado di recuperare i danni che essi hanno procurati. La possibilità di rilevare questi tipi di attacco funge anche da deterrente.

Esistono quattro tipi di attacco:

- **Replay:** i dati raccolti tramite attacchi di tipo passivo vengono riutilizzati in vari modi.
- **Mascheramento:** un'entità non autorizzata si finge un'altra autorizzata. Solitamente necessita l'aver eseguito attacchi passivi in precedenza per raccogliere informazioni.
- **Modifica dei messaggi:** alcune parti di messaggi legittimi vengono alterati, oppure i pacchetti vengono riordinati, ritardati o duplicati per produrre effetti indesiderati.
- **Denial of Service:** si impedisce il normale uso delle risorse sovraccaricando il sistema o interferendo con le comunicazioni.

1.4 Fundamental Security Design Principles – Principi Fondamentali per il Design

Non è possibile disegnare un sistema completamente sicuro, ma esistono una serie di regole che permettono di guidare il design verso soluzioni efficienti.

- **Economia (Economy of mechanism):** è importante mantenere l'hardware e il software che garantiscono le misure di sicurezza *il più semplice possibile*. Un sistema più semplice da testare, da verificare e da mantenere (aggiornare, sostituire, migliorare). Inoltre meno componenti ha il sistema, più bassa è la probabilità che si siano falle di sicurezza.
- **Fail-safe default:** le decisioni riguardanti gli accessi dovrebbero essere basate su permessi piuttosto che esclusione. Di default l'utente non dovrebbe avere permessi, che vengono garantiti in caso di bisogno. In questo modo un errore nella progettazione sarebbe facilmente individuabile e non causerebbe problemi di sicurezza.
- **Complete mediation:** l'esecuzione di ogni operazione dovrebbe essere controllata tramite meccanismi di controllo degli accessi. Non si dovrebbe mai fare affidamento su decisioni memorizzate in una cache. Questo approccio è tuttavia molto pesante per le risorse del sistema, e quindi raramente usato.
- **Open design:** la *security-through-obscurity* è da evitare. È meglio che il sistema di sicurezza sia di pubblico dominio, in modo che molti esperti possano testarlo e valutarlo e che l'utente stesso possa avere fiducia in esso.
- **Separazione dei privilegi (Separation of privilege):** il fatto di richiedere la co-esistenza di più privilegi per permettere l'accesso ad una certa risorsa è un'ottima misura di sicurezza. Un esempio reale è quello dell'autenticazione a più fattori.
- **Privilegi minimi (Least privilege):** ogni processo e ogni utente dovrebbero operare usando il minimo insieme di privilegi richiesti in ogni momento (es. l'amministratore di sistema non dovrebbe continuamente usare l'account root, ma dovrebbe utilizzare un account normale e ottenere i privilegi solo per il periodo richiesto). È utile definire vari ruoli, a cui assegnare diversi permessi per eseguire le proprie funzioni.
- **Minima condivisione possibile (Least common mechanism):** dovrebbero essere minimizzate le funzioni condivise da diversi utenti e l'hardware e il software da cui tutti gli utenti dipendono. Ciò permette di evitare problemi come la race condition nonostante aumenti in parte la complessità del sistema.
- **Accettabilità (Psychological acceptability):** i meccanismi di sicurezza non dovrebbero interferire pesantemente con il lavoro degli utenti (altrimenti l'utente potrebbe decidere di disattivarli oppure potrebbe trovare metodi insicuri per aggirarli), ma allo stesso tempo dovrebbero permettere di soddisfare i requisiti richiesti.
- **Isolamento (Isolation):** si riferisce a 3 contesti:
 - La parte pubblicamente accessibile del sistema dovrebbe essere separata dalla parte contenente risorse e processi critici sia fisicamente (nessuna connessione) che logicamente (strati di servizi di sicurezza).
 - I processi e i file degli utenti dovrebbero essere isolati gli uni dagli altri, a meno che non sia espressamente richiesto.
 - I meccanismi di sicurezza dovrebbero essere isolati dal resto del sistema, impedendo che vi si possa accedere in maniera semplice anche in caso di accesso indebito al sistema (es. utilizzando chiavi crittografiche diverse).
- **Incapsulamento (Encapsulation):** principio che deriva direttamente dalla programmazione OO. Procedure e dati dovrebbero essere isolati in strutture accessibili solamente dal sottosistema che le deve utilizzare tramite specifici punti di accesso (cfr. getter e setter).
- **Modularità (Modularity):** è da incoraggiare l'uso di meccanismi e implementazioni modulari. In questo modo si può risparmiare il tempo per l'implementazione multipla di servizi simili per dedicarlo al miglioramento del singolo meccanismo da sviluppare. La modularità permette inoltre l'adattamento a tecnologie diverse e l'aggiornamento a caratteristiche migliori.
- **Architettura a strati (Layering) e Defense in Depth:** le risorse possono essere protette con più strati sovrapposti, in modo da incrociare le vulnerabilità e rendere più difficile l'accesso alle risorse stesse.
- **Predicibilità (Least astonishment):** ogni programma dovrebbe comportarsi in modo più prevedibile possibile, senza sorprendere l'utente e facendogli capire il motivo dell'adozione di quel meccanismo.

1.5 Attack Surfaces and Attack Tree – Superfici e Alberi di Attacco

Esistono alcuni metodi per classificare le minacce: l'analisi delle Superfici d'Attacco e gli Alberi d'Attacco.

1.5.1 Attack Surfaces – Superfici di Attacco

Parte di sistema raggiungibile che permette di sfruttare alcune vulnerabilità. Tra di esse troviamo:

- Porte aperte verso il mondo tramite server (Web o di altri servizi)
- Servizi disponibili all'interno del firewall aziendale
- Codice che processa dati entranti (email, XML, PDF, ...)
- Interfacce Web e SQL
- Impiegati con accesso a informazioni sensibili e vulnerabili ad attacchi di social engineering

Le superfici possono essere classificate in 3 diverse classi:

- **Network attack surfaces:** si riferiscono a vulnerabilità delle reti (rete aziendale, WAN, Internet), in particolare alle vulnerabilità dei protocolli, sfruttate per attacchi come DoS, interruzione delle comunicazioni, ...
- **Software attack surfaces:** si riferiscono alle vulnerabilità del software (applicazioni, SO, Server Web, ...).
- **Human attack surfaces:** si riferiscono alle vulnerabilità del personale, che potrebbe essere soggetto ad attacchi di social engineering, commettere errori oppure approfittare malevolmente della sua condizione.

È particolarmente utile valutare le superfici di attacco in quanto permettono di comprendere meglio i rischi e i pericoli di ciascun tipo di attacco, quindi dove inserire meccanismi di sicurezza o dove migliorare quelli già esistenti.

1.5.2 Attack Trees – Alberi di Attacco

Strutture dati gerarchiche che rappresentano un insieme di potenziali tecniche in grado di servirsi di vulnerabilità per bucare il sistema. Permettono di comprendere in maniera migliore come vengono condotti i diversi attacchi e quali sono le vulnerabilità chiave. Sono quindi utili sia in fase di design sia in fase di manutenzione o rafforzamento del sistema.

- **Radice:** obiettivo dell'attacco (risorsa da corrompere o danneggiare, parte di sistema a cui accedere, ...)
- **Figli (Foglie e nodi intermedi):** ogni nodo ha un insieme di figli che rappresentano i modi in cui l'attaccante può raggiungere l'obiettivo rappresentato dalla foglia. Potrebbero essere connessi con operatore logico AND (tutte le operazioni devono essere eseguite) oppure OR (è sufficiente eseguirne una sola). Un figlio rappresenta un modo in cui l'attacco può essere iniziato.

1.6 Computer Security Strategy – Strategie della Sicurezza Informatica

Un sistema di sicurezza informatica prevede tre punti chiave: **Specifiche/Politiche** (cosa deve fare quel meccanismo di sicurezza?), **Implementazione/Meccanismo** (come deve farlo?) e **Correttezza/Garanzia** (lo fa veramente?).

1.6.1 Security policy – Politiche di sicurezza

Formalmente vengono definite come un insieme di regole e pratiche che specificano come un sistema dev'essere regolato e organizzato per fornire la sicurezza necessaria a proteggere le risorse critiche del sistema. È necessario considerare diversi fattori:

- **Valore della risorsa da proteggere**
- **Vulnerabilità del sistema**
- **Potenziali minacce e probabilità dell'attacco**
- **Semplicità di utilizzo/Ricadute sulle performance vs Sicurezza:** ogni meccanismo di sicurezza mina in qualche modo la semplicità di utilizzo del sistema e le sue performance. Quali delle due caratteristiche è più importante?
- **Costi della sicurezza vs Costi del danno e del recupero:** il meccanismo di sicurezza ha un certo costo così come lo hanno il danno e le procedure di recupero. Quale è maggiore?

1.6.2 Security Implementation - Implementazioni della sicurezza

- **Prevenzione (Prevention):** uno schema di sicurezza ideale è quello in cui non è possibile eseguire nessun attacco. Esistono molti attacchi che possono essere prevenuti in vari modi (Es. trasmissione di dati cifrati).
- **Riconoscimento (Detection):** la protezione assoluta è irraggiungibile, ma deve almeno essere possibile accorgersi dell'attacco (Es. che individui non autorizzati hanno fatto login).
- **Recupero (Recovery):** possibilità di ripristinare la situazione precedente all'attacco (Es. backup).

1.6.3 Assurance and Evaluation – Garanzia e Valutazione

I fruitori del sistema di sicurezza hanno la necessità di potersi fidare del sistema, sapendo che funziona come previsto e che segua i requisiti richiesti. Chiamiamo **garanzia** (assurance) il grado di sicurezza che si può avere nelle misure di sicurezza tecniche e operative poste in atto. È una valutazione completamente soggettiva e qualitativa.

La **valutazione** (evaluation) è il processo con cui si esamina un prodotto o sistema informatico rispetto a certi criteri in maniera quantitativa, eventualmente utilizzando anche strumenti matematici ed analitici.

2 Cryptographic Tools – Crittografia

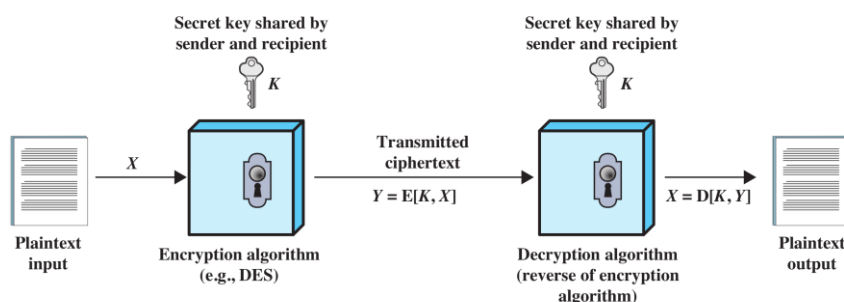
Descrizione dei vari tipi di algoritmi crittografici o correlati alla crittografia: crittografia simmetrica, funzioni hash, crittografia a chiave pubblica.

2.1 Confidentiality with symmetric encryption – Confidenzialità con crittografia a chiave simmetrica

La tecnica più comune per ottenere confidenzialità è la crittografia a chiave simmetrica. Verranno trattati algoritmi a blocchi (DES (Data Encryption Standard) e AES (Advanced Encryption Standard)) e algoritmi a stream.

2.1.1 Symmetric Encryption – Crittografia Simmetrica

Tecnica di crittografia usata per 2000 anni, fino al 1970, quando venne introdotta la crittografia a chiave pubblica.



Uno schema di crittografia a chiave simmetrica si compone di 5 “ingredienti”:

- **Testo in chiaro (Plaintext):** messaggio originale, in chiaro.
- **Algoritmo di crittografia (Encryption algorithm):** esegue sostituzioni e trasformazioni sul testo in chiaro.
- **Chiave segreta (Secret key):** fornita in input all'algoritmo insieme al testo in chiaro, determina come vengono eseguite le trasformazioni e le sostituzioni per produrre il testo crittografato.
- **Testo crittografato (Cipher text):** messaggio modificato prodotto come output dell'algoritmo. Il suo contenuto dipende dal testo in chiaro e dalla chiave simmetrica.
- **Algoritmo di decriptazione (Decryption algorithm):** algoritmo che permette, fornendo in input chiave e ciphertext, di produrre nuovamente il plaintext.

Per il funzionamento di un algoritmo a chiave simmetria sono necessari due requisiti fondamentali:

- L'algoritmo deve essere robusto: non dev'essere semplice ottenere la chiave o decriptare i ciphertext se non si è in possesso della chiave, neanche venendo a conoscenza dei plaintext corrispondenti. La sicurezza di uno schema crittografico dipende solamente dalla qualità dell'algoritmo e non dalla sua segretezza.
- Mittente e ricevente devono potersi scambiare e devono conservare la chiave in maniera sicura. È importante ricordare che nel caso l'attaccante riesca a ottenere la chiave tutti i messaggi crittografati con tale chiave sono compromessi.

Gli attacchi più comuni su crittografia simmetrica sono di due tipi:

- **Crittoanalisi (Cryptoanalysis):** si basa sulla natura dell'algoritmo e sulle conoscenze di caratteristiche generali del plaintext (o ancora meglio su corrispondenze plaintext-chipertext), come per esempio la frequenza delle occorrenze di caratteri in una certa lingua, per dedurre la chiave utilizzata.
- **Attacco brute-force:** vengono provate tutte le possibili chiavi fino a che non si riesce ad ottenere una traduzione intelligibile (è importante ricordare che anche riconoscere quando l'output è intelligibile non è un problema semplice se non se ne conoscono le caratteristiche). Statisticamente la metà delle chiavi disponibili devono essere testate prima di riuscire a decrittografare il messaggio.

2.1.2 Symmetric Block Encryption Algorithm – Algoritmi di crittografia chiave simmetrica a blocchi

Sono gli algoritmi a chiave simmetrica maggiormente utilizzati. Il plaintext viene diviso in blocchi di lunghezza fissa e viene prodotto un output a blocchi di lunghezza fissa (eventualmente diversa da quella in input).

2.1.2.1 Data Encryption Standard (DES)

Introdotta nel 1977, utilizzata ampiamente fino ai giorni nostri.

Caratteristiche:

- Lavora su blocchi lunghi 64 bit
- Chiavi lunghe 56 bit
- Produce blocchi lunghi 64 bit

Potenziati problemi:

- *Riguardo all'algoritmo stesso:* l'algoritmo è stato analizzato per molti anni (è l'algoritmo più studiato) e non sono mai state rilevate potenziali debolezze (es. per attacchi di crittoanalisi). L'algoritmo è però particolarmente efficiente e la lunghezza dei blocchi è limitante.
- *Riguardo alla lunghezza della chiave:* mentre nel 1977 poteva essere sufficiente, oggi la potenza di calcolo dei processori permette di eseguire attacchi a forza bruta parallelizzando la computazione in poche ore.

Sono stati introdotti appositi acceleratori hardware per supportare i computer vecchi nell'esecuzione dell'algoritmo.

2.1.2.2 Triple DES (3DES)

Introdotta nel 1985, utilizzata per estendere la vita dell'algoritmo e degli acceleratori hardware DES. Prevede l'applicazione dell'algoritmo per 3 volte di fila, utilizzando 2 o 3 chiavi diverse (quindi la lunghezza diventa 112 bit o 168 bit). La nuova lunghezza della chiave permette di risolvere il problema di potenziali attacchi brute forcing, ma il problema dell'elevata inefficienza dell'algoritmo non permette di produrre codice efficiente. Permane inoltre il problema dei blocchi troppo piccoli.

2.1.2.3 Advanced Encryption Standard (AES)

Introdotta nel 1997 per rimpiazzare l'obsoleto 3DES.

- Caratteristiche:
 - Blocchi a 128 bit
 - Supporto per chiavi lunghe 128, 192 o 256 bit

Rispetto a DES è decisamente più efficiente e flessibile. Venne scelto in base a un concorso e, come il suo predecessore, è stato ampiamente studiato senza trovare nessun tipo di vulnerabilità ad attacchi crittoanalitici.

2.1.2.4 Practical Security Issues – Problemi di sicurezza degli algoritmi a blocchi

Tipicamente la crittografia a chiave simmetrica viene utilizzata per unità di data più lunghe della lunghezza di un blocco. La modalità più utilizzata è detta **ECB (Electronic CodeBook)** in cui il dato viene diviso in unità di lunghezza accettabile e crittografati tutti con la stessa chiave crittografica. I problemi di questo approccio sono diversi. Per esempio si possono operare attacchi di crittoanalisi basandosi sulla regolarità nella struttura della comunicazione (es. tutti i messaggi iniziano o finiscono nello stesso modo). Inoltre non protegge da attacchi che mirano alla modifica/al riordinamento o alla cancellazione di messaggi. Sono stati introdotti quindi nuovi **mode of operation**.

Un modo per risolvere questo problema è quello di utilizzare **algoritmi a stream**, in cui una coppia di generatori di numeri pseudocasuali (che producono gli stessi numeri su due macchine diverse solo se viene utilizzato come seed la stessa chiave) producono una serie di numeri detti **keystream** che vengono combinati bit a bit con il plaintext utilizzando operazioni di XOR. Con un generatore pseudorandom ben funzionante un algoritmo di questo tipo è tanto sicuro quanto un algoritmo a blocchi con lunghezza di chiave comparabile, ma permette una maggior efficienza e velocità.

In ogni caso entrambi i tipi di algoritmo possono essere utilizzati per qualunque tipo di applicazione.

2.2 Message Authentication and Hash Function – Message Authentication e Funzioni Hash

Mentre la crittografia è adatta a proteggere da attacchi passivi, non è sufficiente per proteggere da attacchi attivi (falsificazioni). È necessario, infatti, utilizzare la message authentication, che permette di garantire l'autenticità del messaggio (sorgente identificata, nessuna modifica durante la trasmissione) e la possibilità di riconoscere se il messaggio è stato ritardato o se lo si sta riusando in un secondo momento per attacchi di tipo replay.

2.2.1 Authentication using Symmetric Encryption – Autenticazione con crittografia a chiave simmetrica

Si potrebbe pensare di garantire l'autenticità di un messaggio criptandolo con un algoritmo a chiave simmetrica. Se, infatti, solo mittente e destinatario condividono tale chiave, allora per forza un messaggio crittografato in un certo modo è stato prodotto da quel mittente. Si potrebbero includere anche un error-detection code per scongiurare la presenza di modifiche e un timestamp per verificarne l'ora di produzione/invio. Questo sistema non è tuttavia sufficiente, poiché non garantisce che i messaggi, per esempio, non vengano riordinati.

2.2.2 Message Authentication without Message Encryption – Autenticazione senza crittografia

Nella maggior parte degli algoritmi per la message authentication viene aggiunto un authentication tag al messaggio. Il messaggio, tuttavia, non è necessariamente crittografato (quindi non è garantita la confidenzialità). Ciò è un vantaggio in alcuni casi, tra cui:

- Messaggi condivisi in maniera broadcast tra più destinazioni: solo una può controllare l'autenticità del messaggio (altrimenti tutte dovrebbero decriptarlo) ed eventualmente avvisa le altre della non autenticità. Si garantisce in questo modo un risparmio di tempo/risorse e una maggiore affidabilità.
- Messaggi scambiati tra componenti che eseguono compiti molto pesanti: non è necessario decriptare tutti i messaggi, ma si può fare il controllo una volta ogni tanto, casualmente.
- Programmi il cui codice è crittografato: non è necessario decriptare tutto il programma ogni volta, ma semplicemente si può controllare la sua autenticità nei momenti in cui è necessario.

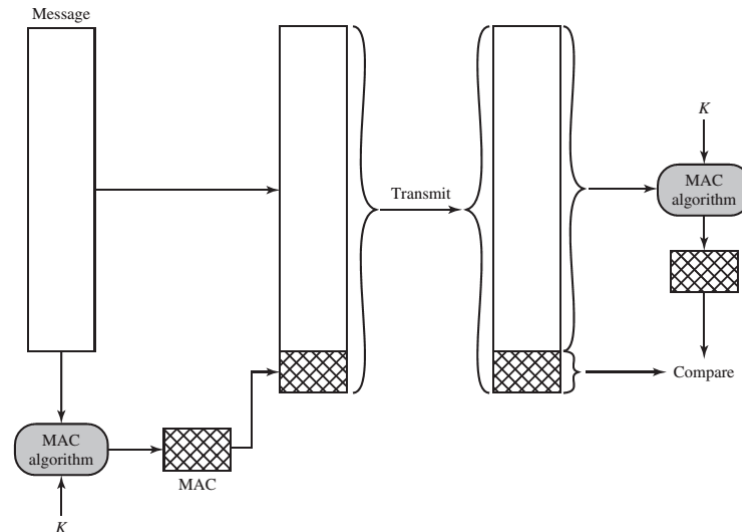
2.2.3 Message Authentication Code

Si utilizza una funzione per produrre un blocco di bit detto *Message Authentication Code (MAC)* che viene accodato al messaggio. La funzione necessita di una chiave condivisa da usare come input per la generazione. Il mittente produce il MAC e lo appende al messaggio, mentre il destinatario, utilizzando la stessa chiave, produce a sua volta il MAC del messaggio ricevuto e lo confronta con quello inviatogli dal mittente. Se non combaciano, allora il messaggio è stato modificato.

Un algoritmo utilizzato per produrre il MAC è DES: si crittografa il messaggio e si usano i 16-32 bit finali del ciphertext. Si potrebbero anche usare algoritmi diversi, ricordando che non è necessaria la reversibilità della funzione.

In questo modo:

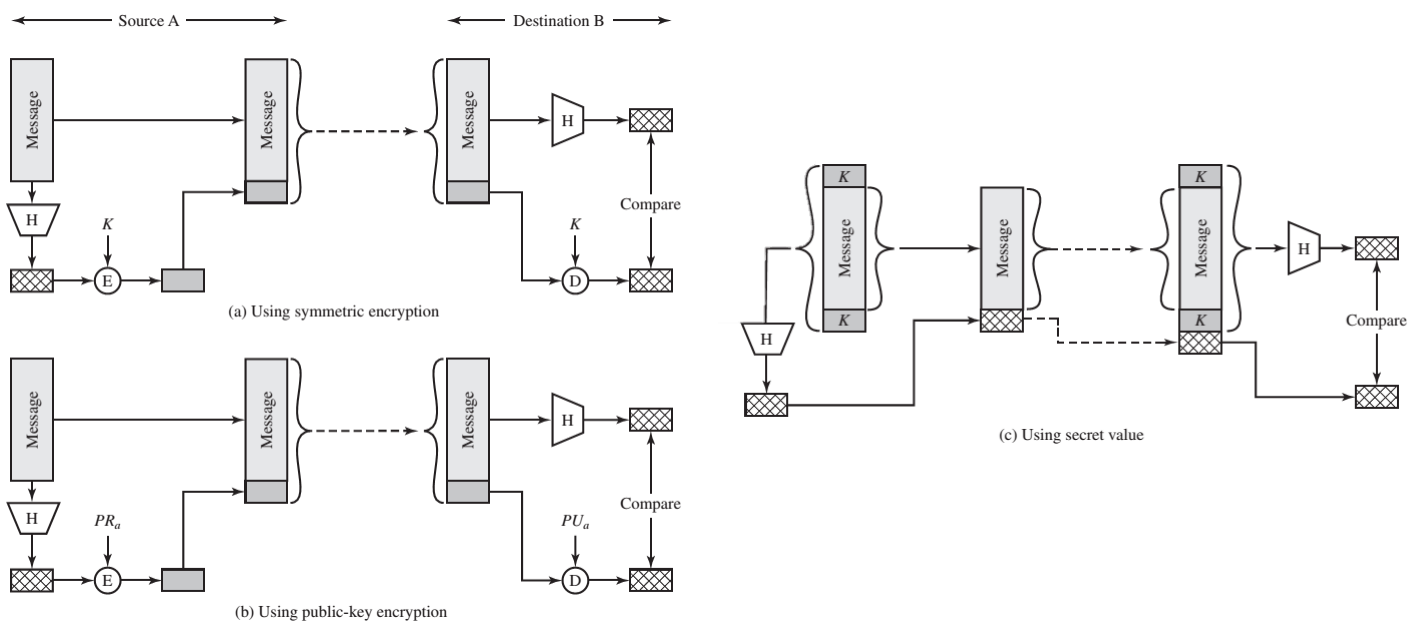
- Il ricevente è sicuro che il messaggio non è stato modificato: l'attaccante non conosce la chiave e non può quindi rigenerare il MAC trasmesso, che sarebbe quindi diverso da quello ricalcolato (in realtà, poiché la lunghezza del MAC è fissa e limitata rispetto a quella del messaggio, si potrebbe costruire un messaggio diverso che produce lo stesso MAC).
- Il ricevente è sicuro che il messaggio è stato inviato da quel mittente, poiché è l'unico che conosce la chiave.
- Se è presente un numero di sequenza, il ricevente può anche essere sicuro dell'ordine poiché il numero non può essere modificato senza ripercussioni sul MAC.



2.2.4 One-Way Hash Function – Funzioni Hash

Una funzione hash prende in input un messaggio di una certa lunghezza e produce un digest di lunghezza fissa. Tipicamente viene aggiunto del padding al messaggio in input fino a renderlo multiplo di una certa dimensione e tra i bit aggiunti si inserisce la lunghezza del messaggio: in questo modo l'attaccante ha il vincolo della lunghezza nel produrre un messaggio modificato con lo stesso digest.

La funzione hash è che non necessita di una chiave segreta in input. Il digest deve tuttavia essere crittografato (per esempio con un algoritmo a chiave simmetrica o meglio con un algoritmo a chiave pubblica che provvede anche una firma digitale) per permettere l'effettiva autenticità. In questo modo è richiesto uno sforzo computazionale minore (non si crittografa l'intero messaggio ma solo il digest, la funzione hash è molto più semplice di una funzione crittografica).



In alternativa alla crittazione del digest si può utilizzare una chiave condivisa (K) da appendere all'inizio e alla fine del messaggio (M). Il mittente produce una sequenza K-M-K e ne produce il digest. Invia poi il messaggio accodando il digest così ottenuto. Il ricevente, essendo a conoscenza di K, può ricostruire la sequenza e ricalcolare il digest. La presenza della chiave sia all'inizio che alla fine del messaggio rende più robusto l'algoritmo. Lo svantaggio è che è necessario condividere un segreto (mentre non serve crittografando il digest con un algoritmo a chiave privata).

2.2.5 Secure Hash Function – Funzioni Hash sicure

Lo scopo di una funzione hash è quella di produrre una sequenza di bit in grado di identificare univocamente un certo messaggio. Dovrebbe avere 6 caratteristiche:

1. Possibilità di essere applicata a un messaggio di qualunque lunghezza.
2. Output di lunghezza fissa.
3. Facilità di computazione per ogni valore (con possibilità di buone implementazioni hardware e software).
4. **One way**: difficoltà dell'operazione per ricavare il messaggio a partire dall'hash.
5. **Weak collision resistance**: difficoltà nel trovare un messaggio y diverso da quello inviato x che abbia lo stesso risultato. (Non posso trovare un messaggio alternativo ad uno che conosco).
6. **Strong collision resistance**: difficoltà nel reperire coppie di messaggi diversi che abbiano lo stesso risultato. (Non posso trovare due messaggi con stesso risultato, quindi diventa impossibile inviare un messaggio con un certo contenuto e sostenere di averne inviato uno diverso solo perché entrambi hanno lo stesso digest).

2.2.5.1 Security of Hash Functions – Sicurezza delle Funzioni Hash

I possibili attacchi ad una funzione hash sono di crittoanalisi (sfruttando debolezze nell'algoritmo stesso) o di tipo brute force (la cui difficoltà dipende solamente dalla lunghezza del digest).

2.2.5.2 Secure Hash Function Algorithms

Le funzioni hash più utilizzate per la message authentication appartengono alla famiglia Secure Hash Algorithm (SHA):

- **SHA**: nata nel 1993, sono state riscontrate alcune debolezze.
- **SHA-1**: nata nel 1995, produceva hash lunghi 160 bit, dismessa poiché non sicura.
- **SHA-2**: nata nel 2002 e diffusa in 3 varianti (SHA-256, SHA-384 e SHA-512) con diversa lunghezza dell'hash, dismessa in quanto non sicura a causa delle similarità con le versioni precedenti nonostante particolarmente sicura in caso di attacchi brute force (in particolare nella versione SHA-512).
- **SHA-3**: nata nel 2012, notevolmente diversa dalla precedente versione.

2.2.6 Other applications of Hash Function – Altre applicazioni delle Funzioni Hash

- Message Authentication
- Firme digitali
- Password (viene memorizzato l'hash della password piuttosto che la password stessa, sono particolarmente importanti la non invertibilità e la strong collision resistance).
- Intrusion detection (si calcola e memorizza l'hash di tutti i file per poterlo ricalcolare e confrontare in un secondo momento, accorgendosi di eventuali modifiche. È importante la weak collision resistance).

2.3 Public-Key Encryption - Crittografia a Chiave Pubblica

Tanto importante quanto quella a chiave simmetrica, utilizzata prevalentemente per la message authentication e la distribuzione di chiavi di sessione.

2.3.1 Public-Key Encryption Structure – Struttura della Crittografia a Chiave Pubblica

Proposta per la prima volta da Diffie e Hellman nel 1976, si basa su funzioni matematiche complesse e rappresenta la prima, vera, innovazione della crittografia in millenni di storia. Il suo funzionamento si basa su una coppia di chiavi, una pubblica e una privata.

Esistono alcuni luoghi comuni da sfatare:

- La crittografia a chiave privata non è più immune di quella a chiave pubblica ad attacchi crittoanalitici. La sicurezza dell'algoritmo dipende solamente dalla lunghezza della chiave e dalla difficoltà nel ricavare informazioni dal ciphertext.
- La crittografia a chiave privata non soppianderà la crittografia a chiave pubblica. Ha infatti un costo computazionale particolarmente elevato.
- La crittografia a chiave privata è più semplice da gestire poiché non necessita di protocolli per produrre e scambiare le chiavi.

Un algoritmo a chiave asimmetrica si basa su 5 componenti:

- **Testo in chiaro (Plaintext)**
- **Algoritmo di crittografia (Encryption algorithm)**
- **Coppia di chiavi pubblica e privata (Public and private key)**: coppia di chiavi utilizzate l'una per criptare e l'altra per decriptare che determinano le precise trasformazioni che l'algoritmo esegue sul plaintext.

- **Testo criptato (Ciphertext):** messaggio trasformato prodotto in output dall'algoritmo. Dipende dalla chiave e dal contenuto del messaggio.
- **Algoritmo di decriptazione (Decryption algorithm)**

2.3.2 Applications for Public-Key Cryptosystems - Applicazioni della Crittografia a Chiave Pubblica

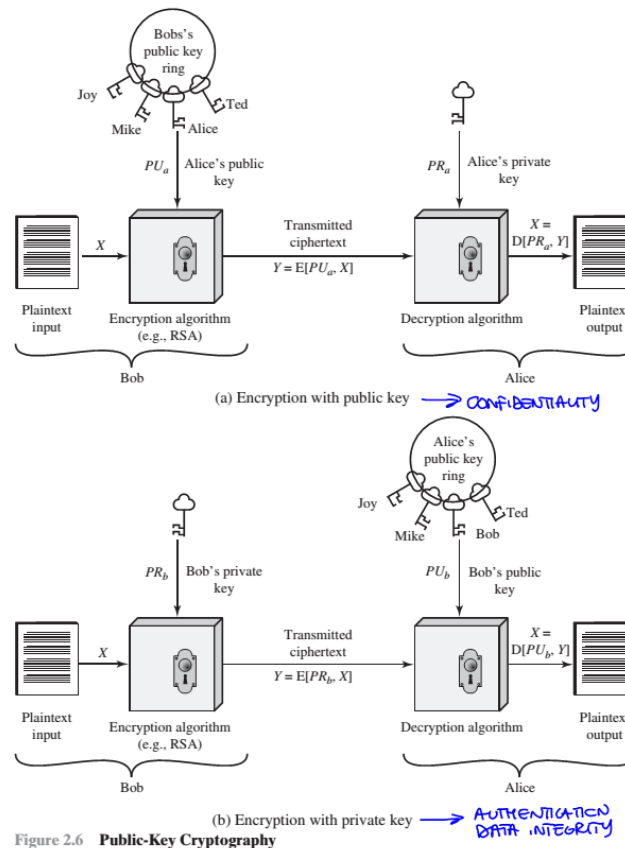


Figure 2.6 Public-Key Cryptography

La crittografia a chiave pubblica ha 3 principali modi di utilizzo:

- **Criptazione con chiave pubblica del destinatario, decriptazione con chiave privata del destinatario (A):** garantisce confidenzialità (solo il destinatario può decriptare il messaggio), utilizzata per esempio per la distribuzione di chiavi di sessione.
- **Criptazione con chiave privata del mittente, decriptazione con chiave pubblica del mittente (B):** garantisce autenticazione e/o integrità dei dati (il destinatario può decriptare il messaggio con una certa chiave pubblica, quindi il messaggio può essere stato crittografato solo da colui che possiede la relativa chiave privata), utilizzata per esempio come firma digitale.
- **Entrambi:** garantisce entrambi i vantaggi, utilizzata per esempio per la criptazione di chiavi segrete.

2.3.3 Requirements for Public-Key Cryptography – Requisiti per la Crittografia a Chiave Pubblica

Diffie e Hellman hanno teorizzato l'esistenza di algoritmi di questo tipo senza però realizzarli fisicamente definendo 5/6 caratteristiche che dovrebbero avere:

1. Il destinatario può generare facilmente una coppia di chiavi (PU_b e PR_b).
2. Il mittente può, conoscendo messaggio M e chiave pubblica PU_b , calcolare il ciphertext $C = E(PU_b, M)$.
3. Il destinatario può decriptare facilmente il ciphertext utilizzando la sua chiave privata $M = D(PR_b, C)$.
4. Un attaccante non può determinare la chiave privata PR_b conoscendo la chiave pubblica PU_b .
5. Un attaccante non può determinare il messaggio M conoscendo il ciphertext C e la chiave pubblica PU_b .
6. (Opzionale) Le due chiavi possono essere utilizzate sia per criptare che per decriptare messaggi.

2.3.4 Asymmetric Encryption Algorithm – Algoritmi di Crittografia a Chiave Asimmetrica

2.3.4.1 RSA

Uno dei primi algoritmi di crittografia a chiave asimmetrica fu RSA, progettato da Ron Rivest, Adi Shamir e Len Adelman nel 1977. Esso è tutt'ora l'algoritmo più utilizzato, implementato e studiato. Originalmente prevedeva chiavi da 129 cifre decimali (428 bit). Oggi si considera virtualmente sicuro RSA-1024 (1024 bit, 300 cifre decimali).

2.4 Digital Signatures and Key Management – Firme Digitali e Gestione delle Chiavi

La crittografia asimmetrica è usata in diversi campi, tra cui le firme digitali e la gestione delle chiavi (distribuzione delle chiavi pubbliche, distribuzione di chiavi simmetriche, distribuzione di chiavi di sessione).

2.4.1 Digital Signatures – Firme Digitali

La crittografia asimmetrica può essere utilizzata per l'autenticazione. Il metodo usato è il seguente:

1. Il mittente allega al messaggio l'hash del messaggio stesso, criptandolo con la propria chiave privata. Esso prende il nome di **firma digitale**.
2. Il ricevente calcola a sua volta l'hash del messaggio ricevuto e lo compara con quello ricevuto insieme al messaggio dopo averlo decriptato con la chiave pubblica del mittente. Se coincidono, è sicuro che:
 - Il messaggio non è stato modificato (altrimenti l'hash sarebbe cambiato)
 - Il messaggio proviene dal mittente che possiede la chiave privata correlata alla chiave pubblica usata (essendo la chiave privata segreta, solo lui avrebbe potuto creare un hash decriptabile con tale chiave pubblica).

È importante ricordare che la chiave pubblica non garantisce confidenzialità. Infatti solo una parte del messaggio è crittografata, ma anche se l'intero contenuto fosse criptato tutti potrebbero decriptarlo utilizzando la chiave pubblica del mittente.

2.4.2 Public-Key Certificates – I Certificati per le Chiavi Pubbliche

Il punto di forza di algoritmi a chiave pubblica è che non c'è necessità di condividere segreti, in quanto la chiave pubblica è, appunto, pubblica. Tuttavia anche questo approccio causa problemi: un attaccante potrebbe infatti condividere una chiave pubblica impersonando una vittima. In questo modo potrebbe quindi riuscire a leggere messaggi non destinati a lui.

La soluzione a questo sistema è la certificazione delle chiavi pubbliche: la chiave viene firmata da una parte terza di cui ci si fida (Certification Authority, CA). Il procedimento avviene in questo modo:

1. L'utente produce una coppia di chiavi: l'una pubblica, l'altra privata
2. L'utente prepara il certificato, inserendo al suo interno il suo identificatore e la chiave pubblica.
3. L'utente invia il certificato vuoto alla CA perché venga firmato. Questa operazione deve essere fatta in maniera sicura (es. face-to-face).
4. La CA calcola l'hash del certificato e lo cripta con la propria chiave privata. Aggiunge il risultato al certificato.

Chiunque voglia verificare l'autenticità della chiave pubblica non deve fare altro che calcolarne l'hash e confrontarlo con quello allegato al certificato dopo averlo decriptato con la chiave pubblica della CA.

2.4.3 Symmetric Key Exchange Using Public-Key Encryption – Condivisione di Chiavi Simmetriche

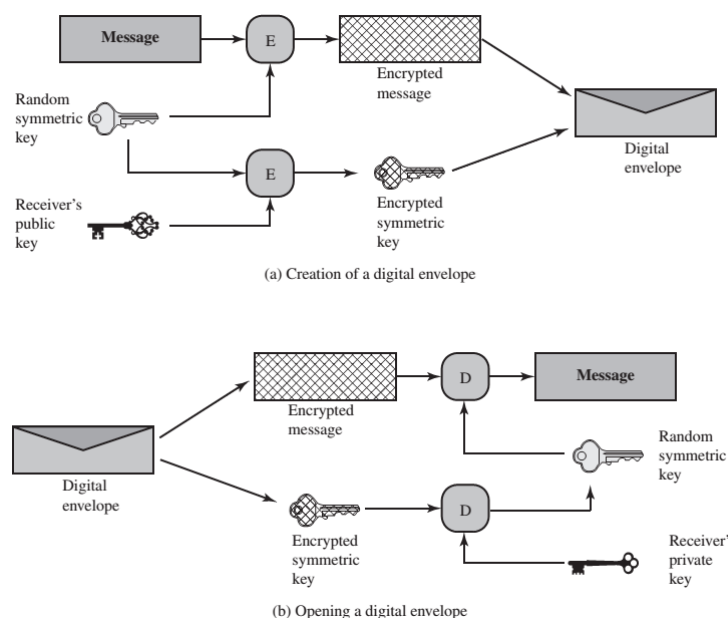
Con la crittografia simmetrica uno dei requisiti fondamentali è quello di riuscire a scambiare la chiave in maniera sicura.

Gli approcci principali sono due: Diffie-Hellman key exchange (che tuttavia non prevede autenticazione) e l'uso di Digital Envelopes.

2.4.4 Digital Envelopes

Sistema utilizzato per scambiare chiavi simmetriche di sessione. Funziona nel modo seguente:

1. Viene generata casualmente una chiave simmetrica di sessione.
2. Il messaggio viene criptato con tale chiave simmetrica.
3. La chiave simmetrica viene criptata con la chiave pubblica del destinatario.
4. Il pacchetto contenente chiave e messaggio viene inviato al destinatario, unico in grado di decriptare la chiave simmetrica utilizzando la sua chiave privata e, di conseguenza, potrà decriptare il messaggio.



2.5 Random and Pseudorandom Number – Numeri Casuali e Pseudocasuali

2.5.1 The Use of Random Numbers - L'uso di Numeri Casuali

I numeri casuali vengono impiegati in svariati campi:

- Generazione di chiavi (pubbliche/private e simmetriche)
- Generazione di chiavi per stream cipher simmetrici
- Generazione di chiavi di sessione o creazione di digital envelopes
- Handshaking per la prevenzione di attacchi replay

2.5.1.1 Randomness - Casualità

Un buon generatore di numeri casuali dovrebbe avere due caratteristiche:

- **Distribuzione uniforme:** la frequenza di occorrenza di ogni numero dovrebbe essere approssimativamente la stessa.
- **Indipendenza:** nessun valore della sequenza dovrebbe poter essere individuato a partire dagli altri.

2.5.1.2 Unpredictability – Casualità

Nonostante siano entrambi importanti, la seconda caratteristica è cruciale: ogni numero dovrebbe essere statisticamente indipendente dagli altri numeri precedentemente estratti, quindi imprevedibile. In questo modo un attaccante non può essere in grado di prevedere i prossimi numeri estratti basandosi su quelli precedenti.

2.5.2 Random versus Pseudorandom

Esistono due tipi di generatori di numeri casuali:

- **True Random Number Generator:** utilizzano una sorgente non deterministica (processi naturali come scarica dei gas, condensatori in perdita, eventi di radiazione ecc.) per produrre casualità. Necessitano di appositi co-processori.
- **Pseudorandom Number Generator:** utilizzano algoritmi deterministici per produrre sequenze deterministiche di numeri. Essa dipende dal *seed*, ovvero un input che viene fornito e che determina la sequenza che verrà estratta. Esso può essere scelto da sorgenti con un basso grado di casualità (es. pressione di tasti, posizione del mouse, millisecondo, ...).

2.6 Practical Application: Encryption of Stored Data – Applicazione pratica: memorizzazione di dati criptati

Nella memorizzazione dei dati devono essere previsti diversi meccanismi di protezione, tra cui controllo degli accessi, intrusion detection e intrusion prevention. Bisogna inoltre considerare che tutti gli approcci utilizzati potrebbero diventare vulnerabili a causa di fattori umani.

Un esempio di servizio di crittografia per dati è PGP (Pretty Good Privacy). Esso permette all'utente di generare una chiave a partire da una password e di utilizzarla per crittografare i file su disco. Per recuperare il file l'utente dovrà inserire la password, da cui PGP genererà la chiave che utilizzerà per decifrare il dato richiesto. Approcci di questo tipo vengono utilizzati per:

- **Applicazioni back-end:** dispositivo hardware posto tra i server di memorizzazione che si occupa di criptare i dati verso il server e decriptare i dati verso l'esterno con una velocità più alta possibile, cercando di introdurre la minima latenza possibile.
- **Crittografia di nastri:** un co-processore incorporato all'interno del drive che legge il nastro permette di crittografare e de-crittografare i dati memorizzati all'interno del nastro utilizzando un'apposita chiave. La chiave è integrata nell'hardware, ma eventualmente può essere estratta per essere trasportata o inviata in maniera sicura.
- **Crittografia dei dati su disco:** un software si occupa di crittografare i file contenuti all'interno del disco (tutti o quelli contenuti in certe cartelle). Gli approcci sono due: crittografare direttamente i dati oppure creare un disco virtuale, salvato localmente sull'hard disk fisico, in cui mantenere i dati criptati.

3 User Authentication

L'identificazione è il processo con cui l'utente si dichiara sé stesso al sistema. Il processo di user authentication permette di verificare la veridicità di tale dichiarazione. Attenzione: non è da confondere con la message

authentication, che è la procedura che permette di verificare che il contenuto di un messaggio non sia stato alterato e che il mittente e il destinatario siano veri.

3.1 Electronic User Authentication Principles – I principi della User Authentication

L'electronic user authentication consiste nel processo che determina se le credenziali fornite elettronicamente da un utente siano valide o meno e le usa, per esempio, per permettere l'accesso a una certa risorsa o l'esecuzione di una certa operazione. Spesso l'autenticazione avviene in forma remota (per esempio attraverso la rete internet).

3.1.1 A Model for Electronic User Authentication – Un modello per la User Authentication

Un requisito fondamentale è che l'utente sia registrato al sistema. La registrazione avviene in questo modo:

- L'utente richiede a una **Registration Authority (RA)** di potersi registrare al **Credential Service Provider (CSP)**. La RA verificherà le credenziali e trasmetterà l'identità al CSP.
- Il CPS rilascia delle **credenziali** all'utente. Le credenziali sono dati che collegano un'identità (e altri attributi) a un token in possesso del/conosciuto dall'utente. Il token può essere creato e consegnato dal CSP oppure costruito basandosi su informazioni fornite dall'utente.

La procedura di autenticazione avviene poi in questo modo:

- L'utente fornisce informazioni ad un **verificatore** che ne verifica la veridicità e passa il risultato dell'operazione a una **Relying Party (RP)**.
- La RP utilizza le informazioni ricevute per eseguire operazioni di access control o per garantire autorizzazioni.

3.1.2 Means of Authentication – Metodi di Autenticazione

L'autenticazione può basarsi su:

Metodo	Descrizione	Problemi
<i>Qualcosa che l'utente conosce</i>	Password, numero di identificazione personale, PIN, risposte a domande, ...	• L'utente può dimenticare il token • Un attaccante può tentare di indovinarlo
<i>Qualcosa che l'utente possiede</i>	Chiave, smart card, ... Il mezzo per eseguire l'autenticazione è detto <i>token</i> .	• L'utente può perdere il token • Un attaccante può clonarlo
<i>Qualcosa che l'utente è</i>	Biometrica (impronta digitale, retina, faccia, ...)	• Costo dell'apparecchiatura • Alta probabilità di falsi negativi o falsi positivi
<i>Qualcosa che l'utente fa</i>	Biometrica dinamica (pattern vocale, scrittura, velocità di digitazione, ...)	

3.2 Password-Based Authentication – Autenticazione con Password

Il più comune metodo di autenticazione è ottenuto con la coppia nome utente/password: l'utente fornisce al sistema un identificatore (pubblico) e una password. Il sistema compara la password fornita con quella associata all'identificatore in un apposito file/database delle password e determina se l'individuo è autorizzato ad accedere al sistema (chi è autorizzato, quali privilegi ha).

3.2.1 The Vulnerability of Password - Vulnerabilità delle Password

Di seguito verranno elencati gli attacchi che si possono attuare a un sistema con autenticazione basata su password. Tipicamente un sistema di questo tipo memorizza le password in un file, ordinate per user ID. Esse possono essere salvate in chiaro oppure, più spesso, viene memorizzato il loro hash (forma più sicura).

Attacco	Descrizione	Contromisura
<i>Attacco a dizionario offline</i>	L'attaccante ottiene l'accesso al file delle password e compara gli hash che vi si trovano all'interno con quelli di password comuni.	• Access control per prevenire la lettura del file delle password • Intrusion detection • Cambiamento immediato delle password compromesse
<i>Attacco ad un account specifico</i>	L'attaccante tenta di indovinare la password di un certo account.	• Lockout: dopo un certo numero di accessi l'account viene bloccato.

<i>Attacco basato su password popolari</i>	L'attaccante effettua tentativi con password popolari per accedere a uno dei tanti utenti del sistema.	- Politiche per impedire l'uso di password comuni - Verifica dell'indirizzo IP dei richiedenti
<i>Password guessing di uno specifico utente</i>	L'attaccante entra a conoscenza di informazioni che gli permettono di scoprire la password di un utente.	Politiche per la scelta di password più severe: segretezza, lunghezza minima, character set, obbligo di cambiamento dopo un certo tempo, ...
<i>Workstation hijacking</i>	L'attaccante si impossessa di un computer che l'utente ha dimenticato connesso in sua assenza.	- Log-out automatico - Intrusion detection (il comportamento sarà diverso da quello dell'utente reale)
<i>Errori dell'utente</i>	L'attaccante fa in modo che l'utente gli condivida la password, utilizza tecniche di social engineering per scoprirla oppure utilizza la password di default che l'utente non ha cambiato.	- Training - Intrusion detection - Autenticazione a più fattori (password più semplice → l'utente non la scrive sul post-it)
<i>Sfruttamento del riuso di password</i>	L'attaccante sfrutta il fatto che la password è condivisa tra più servizi.	Impedire l'uso della stessa password o di password simili
<i>Electronic monitoring</i>	L'attaccante intercetta la password mentre essa viene trasmessa.	Crittografia avanzata

Nonostante gli svariati problemi, questo metodo è ancora il più utilizzato in quanto non richiede l'implementazione di particolari servizi, è economico (rispetto a sistemi biometrici), permette di utilizzare una password diversa per ogni servizio (impossibile con sistemi biometrici, costoso con smartcard o altri sistemi simili).

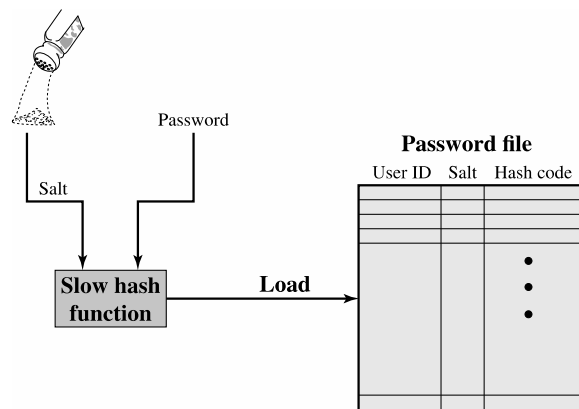
3.2.2 The Use of Hashed Passwords – Hash delle Password

Uno dei sistemi per rendere più sicure le password memorizzate è quello di memorizzare l'output di una funzione hash a cui viene inserito in input la password stessa e un valore di **sale (salt)**. Esso viene generato in qualche modo (casualmente, basandosi sull'orario, ecc.) al momento della registrazione.

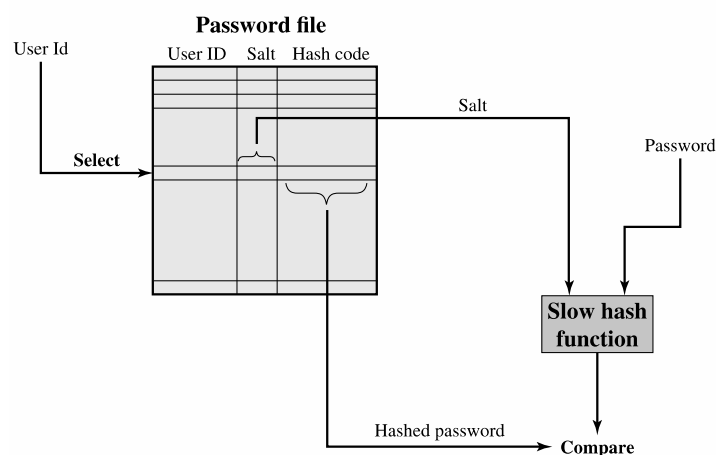
L'introduzione del sale ha diversi vantaggi:

- I duplicati non sono più direttamente visibili (sia password uguali per utenti diversi sullo stesso sistema che password uguali per lo stesso utente su diversi sistemi).
- Aumenta esponenzialmente la difficoltà di attacchi a dizionario: un eventuale attaccante non solo deve confrontare l'hash di ogni password con il valore memorizzato per riuscire a scoprire qual è la parola corrispondente, ma per ogni password deve provare anche tutti i valori di sale. Inoltre spesso la funzione hash è specificatamente progettata per essere lenta e quindi scoraggiare gli attacchi.

Esistono comunque alcune vulnerabilità legate alla possibilità di clonare il file delle password ed eseguire il cracking in maniera asincrona.



(a) Loading a new password



(b) Verifying a password

3.2.2.1 UNIX Implementations – Implementazione UNIX

L'implementazione classica UNIX prevede l'adozione di questa procedura:

- L'utente seleziona una password lunga fino a 8 caratteri, convertita in un valore a 56 bit (7-bit ASCII).
- Viene generato un valore di sale a 12 bit.
- Una funzione hash basata su DES detta *crypt* (one way, lenta) prende in input la password e il sale e viene eseguita su un blocco di 64 zeri. Questa operazione viene ripetuta 25 volte. L'output è una sequenza di 11 caratteri.

Questo schema, tuttavia, non è particolarmente sicuro a causa della maggior potenza dei processori odierni. Le migliorie utilizzate oggi in Linux e Solaris consistono nell'uso di una funzione hash basata su MD5 ripetuta 1000 volte, un valore di sale da 48 bit e un hash risultante lungo 128 bit. OpenBSD utilizza invece un algoritmo di crittografia basato su Blowfish permettendo password lunghe fino a 55 caratteri, sale da 128 bit e output risultante a 192 bit.

3.2.3 Password Cracking of User-Chosen Passwords – Cracking di Password

3.2.3.1 Traditional Approach – Approcci classici

La maniera più semplice per scoprire la password di un utente è quella di creare un grande dizionario con tutte le possibilità, calcolarne gli hash con ogni possibile valore di sale e confrontarli con quelli memorizzati nel file. La maggior parte degli utenti, infatti, utilizzano password banali. È possibile “scambiare” tempo con spazio costruendo una **rainbow table**, ovvero pre-calcolando tali valori salvandoli in una tabella.

È da notare che la lunghezza di una password non serve a nulla se è banale (es. nome, indirizzo, parola di dizionario, ecc.) in quanto il password cracker non fa altro che provare combinazioni di parole prima di passare, in caso di insuccesso, a testare tutti i caratteri.

3.2.3.2 Modern Approach – Approccio attuale

Oggi grazie a politiche di sicurezza e training gli utenti scelgono password migliori, ma allo stesso tempo la potenza di calcolo è notevolmente aumentata. Rimane quindi abbastanza semplice scoprire la password, anche basandosi sui dati ottenuti analizzando i file delle password diffusi in passato. In rete sono disponibili file in cui vengono elencate tutte le password più comuni, in ordine di utilizzo.

3.2.4 Password File Access Control – Controllo degli accessi al file delle password

Uno dei modi più efficaci per proteggere le password è impedire l'accesso ai non utenti e agli utenti non privilegiati al file delle password. In questo modo per poter accedere al file è necessario conoscere almeno la password di un utente. Spesso addirittura le password (hashate) vengono mantenute in un file separato da quello che specifica gli utenti del sistema chiamato **shadow password file**.

Un attaccante può tuttavia riuscire ad accedere al file in diversi modi:

- Sfruttando una vulnerabilità nel sistema operativo o del controllo degli accessi.
- Sfruttando un incidente che renda leggibile il file per un certo tempo.
- Ottenendo la password di un utente perché utilizzata in altri sistemi non particolarmente sicuri.
- Accedendo fisicamente al supporto di memorizzazione o a un supporto di backup.
- Intercettandole mentre vengono trasferite tramite la rete.

3.2.5 Password Selection Strategy - Strategie di selezione della password

Quando non obbligati, gli utenti tendono a scegliere password troppo semplici da indovinare o troppo brevi. È tuttavia da considerare che un utente potrebbe non essere in grado di memorizzare password lunghe e composte da caratteri completamente casuali (quindi non indovinabili e sufficientemente lunghe per rendere inefficace o comunque complesso un attacco brute force). Le seguenti tecniche permettono di evitare l'utilizzo di password non sufficientemente sicure:

- **Educazione dell'utente:** si insegna all'utente come creare una password robusta. Essi possono, tuttavia, ignorare le linee guida o semplicemente non essere in grado di applicarle in maniera corretta.
- **Password generate dal computer:** le password vengono generate casualmente. Sono tuttavia complesse da memorizzare e spesso gli utenti le appuntano da qualche parte, rendendo insicuro il meccanismo.
- **Reactive password checking:** ad intervalli regolari viene eseguito un password cracker che evidenzia le password utilizzate ma non sicure. È tuttavia costoso a livello di tempo e risorse.

- **Politiche per password complesse oppure controllo proattivo delle password:** la password viene controllata al momento della selezione e, in caso non sia sufficientemente complessa viene immediatamente rigettata. È necessario valutare il bilanciamento tra user acceptability e robustezza della password. Le regole generalmente imposte dagli amministratori sono:
 - Lunghezza superiore a X caratteri (Es. 8)
 - Presenza di almeno una lettera maiuscola, una minuscola, un numero, un simbolo di interpunzione ecc.

In alternativa si può vietare l'uso di una serie di password considerate fragili memorizzate in un grande database. Ciò ha due risvolti negativi: occupa molto spazio su disco e, inoltre, la singola interrogazione potrebbe impiegare molto tempo.

3.3 Token-Based Authentication – Autenticazione basata su Token

Oltre all'autenticazione con password, spesso si ricorre all'autenticazione con token. Un token è un oggetto in possesso dell'utente (Es. memory card o smart card) che deve essere utilizzato nella procedura di autenticazione.

3.3.1 Memory Cards

Supporti in grado di memorizzare (ma non processare) dati. Un esempio sono le carte con banda magnetica, che possono essere programmate e lette da appositi lettori. Esse possono essere usate in due modi:

- Autonomamente: sistema insicuro, è sufficiente che qualcuno rubi o cloni il token per avere l'accesso.
- In accoppiata con un segreto (Es. PIN): sistema più sicuro in quanto un attaccante deve, oltre che entrare in possesso del token, scoprire il segreto.

I problemi di un sistema di questo tipo sono:

- **Necessità di appositi lettori:** il costo per l'utilizzo del token aumenta in quanto serve dell'hardware apposito.
- **Perdita dei token:** il token può essere perso, causando un costo per la sostituzione e potenziali problemi di sicurezza nel caso venga usato da altri individui.
- **Insoddisfazione dell'utente:** il sistema potrebbe non essere particolarmente apprezzato dagli utenti.

3.3.2 Smart Cards

A differenza delle memory card, contengono un **microprocessore** (più eventualmente co-processori crittografici) e **memorie** (non programmabili, e programmabili). Necessitano di apposito **hardware** per essere lette (tramite contatti elettrici oppure contactless come nel caso di RFID o NFC) e per permettere all'utente di interagirvi.

I protocolli di autenticazione utilizzati sono 3:

- **Statico:** l'utente si autentica al token, il token si autentica al computer (come farebbe un token di memoria).
- **Dinamico:** il token genera una password periodica che viene inserita nel sistema (automaticamente o manualmente). Il computer e il token devono essere opportunamente sincronizzati in modo che il sistema conosca in ogni momento la password corrente del token.
- **Challenge-response:** il sistema genera una richiesta (Es. una stringa casuale) a cui il token deve rispondere (per esempio crittografandoli con la sua chiave privata).

3.5 Remote User Authentication

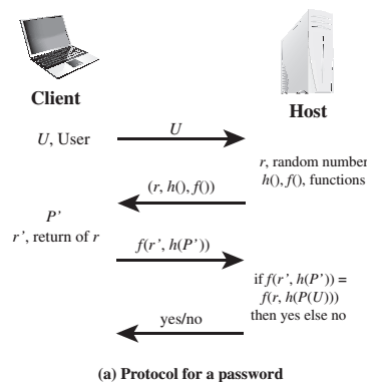
L'autenticazione remota, eseguita tramite rete o altri mezzi di comunicazione, introduce nuovi problemi di sicurezza, come l'eavesdropping e la possibilità di replay.

3.5.1 Password Protocol

È necessario utilizzare appositi sistemi di verifica della password. Un esempio è il **challenge-response**:

- L'utente trasmette la propria identità all'host remoto.
- L'host genera un numero casuale r (**nonce**) a lo invia all'utente insieme a due funzioni, $h()$ e $f()$ (**challenge**).
- L'utente utilizza la funzione $f()$ inserendo come parametri $h(password)$ e r .
- Il sistema esegue lo stesso calcolo e verifica che essi coincidano: se sì, l'utente è autenticato.

Un sistema di questo tipo permette di difendersi da diversi attacchi, tra cui eavesdropping e replay (né la password né il suo hash vengono inviati direttamente, ma si utilizza una funzione di hash e un numero casuale per impedire il riuso dei dati inviati) e furto delle password dall'host (non viene memorizzata la password in chiaro ma il suo hash).



3.6 Security Issues for User Authentication – Problemi di sicurezza nella User Authentication

Attacchi	Metodo di autenticazione	Esempi	Contromisure
Client attack	Password	Guessing, brute force (ricerca esaustiva)	Password con elevata entropia, limitazione del numero di tentativi (lock-out)
	Token	Ricerca esaustiva	Passcode con elevata entropia, lock-out, protezione fisica del token
Host attack	Password	Furto del file delle password, attacchi a dizionario, attacchi esaustivi	Hashing delle password memorizzate, entropia, protezione del file
	Token	Furto del passcode	Hashing dei passcode memorizzati, entropia, protezione del file, 1-time passcode
Eavesdropping, furto, copia	Password	Osservazione fisica dell'utente, keylogging	Diligenza dell'utente nel mantenere segreta la password, diligenza dell'amministratore nel revocare credenziali non valide, autenticazione a più fattori
	Token	Furto, contraffazione dell'hardware	Autenticazione più fattori, token non contraffabili
Replay	Password	Furto e riuso delle password	Challenge-response
	Token	Furto e riuso di passcode	Challenge-response, 1-time passcode
Trojan Horse	Password, token	Installazione di dispositivi hardware contraffatti, installazione di applicazioni malevole	Autenticazione del client, difesa fisica dei dispositivi hardware
Denial of Service (DOS)	Password, token	Lock-out dell'account, saturazione della banda	Autenticazione multifattore con token

4 Access Control – Controllo degli Accessi

- **NIST:** insieme di processi che garantiscono o impediscono richieste specifiche dell'utente per utilizzare informazioni, eseguire operazioni o accedere a luoghi fisici.
- **RFC 4949:** processo con cui l'uso delle singole risorse di sistema è regolato permettendo o negando l'accesso degli utenti in riferimento a specifiche politiche di sicurezza.

4.1 Access Control Principles – Principi del Controllo degli Accessi

Il controllo degli accessi è una componente fondamentale nella gestione della sicurezza di un sistema informatico. Esso implementa politiche che specificano chi (quale utente) o cosa (quale applicazione/processo) può accedere a una certa risorsa.

4.1.1 Access Control Context

L'access control coinvolge 3 diverse fasi:

- **Autenticazione:** verifica che le credenziali di un utente o di un sistema siano valide.
- **Autorizzazione:** fa in modo che vengano forniti i giusti permessi ad ogni entità.
- **Audit:** procedura svolta da entità indipendenti che esaminano le attività e le funzioni del sistema con lo scopo di verificare l'adeguatezza dei sistemi di controllo, la coerenza con le politiche e le procedure definite e per determinare la presenza di falle di sicurezza.

Un meccanismo per il controllo degli accessi media tra l'utente e le risorse del sistema: autentica l'entità, determinando se può o meno accedere al sistema e in seguito determina quali sono i suoi permessi specifici (dati salvati su un apposito database) permettendo o meno l'accesso.

4.1.2 Access Control Policies – Politiche per il Controllo degli Accessi

Esistono 3 sistemi principali per definire il funzionamento del controllo degli accessi:

- **Discretionary Access Control (DAC):** si basa sull'identità dell'utente richiedente e su una serie di regole d'accesso che indicano cosa può e cosa non può fare. Un utente con il giusto permesso su una certa entità potrebbe, a sua volta, modificare i permessi dell'entità per abilitare l'accesso a un altro utente.
- **Mandatory Access Control (MAC):** basata sul confronto di etichette (label, assegnate ad ogni risorsa) e livelli (clearance, assegnate ad ogni utente) di sicurezza. Un utente non è in grado di abilitare l'accesso per un altro utente per nessuna risorsa a cui ha accesso (non può modificare né l'etichetta dell'altro utente né la clearance della risorsa).
- **Role-based Access Control (RBAC):** basato sul ruolo che un utente ha nel sistema e quindi sui permessi connessi ad esso.

Tradizionalmente tutti i sistemi operativi implementano una versione di DAC. Le politiche non sono, tuttavia, mutuamente esclusive: potrebbero adottare anche 2 o più politiche contemporaneamente.

4.2 Subjects, Objects and Access Rights – Soggetti, Oggetti e diritti di accesso

- **Soggetto (Subject):** entità in grado di accedere agli oggetti. Spesso viene equiparato a un processo (non solo l'utente può richiedere l'accesso a una risorsa, ma anche un'applicazione può farlo: entrambi possono essere rappresentati da un processo). Tipicamente è considerato responsabile delle azioni che esegue sul sistema. Generalmente i soggetti vengono classificati in 3 gruppi, ognuno con specifici diritti di accesso:
 - **Proprietario (Owner):** è il creatore di una risorsa. Generalmente è l'amministratore per una risorsa di sistema o il capo progetto per una risorsa appartenente a un progetto.
 - **Gruppo (Group):** insieme di utenti a cui possono essere forniti diritti di accesso per alcune risorse. Per esempio il team di persone che lavora ad un certo progetto. Un utente potrebbe appartenere a più gruppi contemporaneamente.
 - **Altri (World):** comprende tutti gli utenti che non sono creatori e non appartengono al gruppo specifico di utenti a cui sono stati garantiti privilegi maggiori. Secondo la regola del "least privilege", devono essere assegnati loro meno diritti possibili.
- **Oggetto (Object):** risorsa che può essere controllata dagli utenti. Essa può contenere e/o ricevere informazioni. Le risorse sono molto numerose, per cui è importante trovare il giusto compromesso tra finezza della gestione degli accessi e la semplicità di impostazione in modo da non generare un carico di lavoro esagerato.

Possono essere gestiti diversi tipi di accesso ad ogni risorsa:

- **Lettura:** possibilità di visualizzare le informazioni contenute all'interno della risorsa. Include la possibilità di copiare e leggere.
- **Scrittura:** possibilità di aggiungere, cancellare o modificare i dati contenuti all'interno della risorsa. Include l'accesso in lettura.
- **Esecuzione:** accesso per programmi e file eseguibili, indica la possibilità di utilizzarli.
- **Delete:** possibilità di cancellare la risorsa e le informazioni contenute al suo interno.
- **Create:** possibilità di creare nuovi file, record o campi.
- **Ricerca:** possibilità di visualizzare l'elenco dei file di una directory (list) e di cercarli al suo interno.

4.3 Discretionary Access Control

Schema in cui un'entità possiede un certo insieme di diritti di accesso che possono permettere di gestire a sua volta i permessi di una certa risorsa per un'altra entità.

L'approccio più comune per la sua implementazione è la **matrice degli accessi** (access matrix), formulata inizialmente da Lampson e in seguito migliorata da Graham e Denning. In essa:

- Una dimensione identifica i soggetti che possono tentare di accedere alle risorse. Potrebbero essere singoli utenti, gruppi, applicazioni, ecc.
- L'altra dimensione identifica gli oggetti che possono essere acceduti dagli utenti. Essi potrebbero essere singoli file oppure gruppi aggregati (record, interi database, ...).
- Ogni cella della matrice è valorizzata con il tipo di accesso che un soggetto ha per un particolare oggetto.

Poiché la matrice è sparsa e molto grande, non può essere mantenuta in memoria tutta unita. Ci sono due tipi di decomposizione:

- Per colonne: ogni colonna viene detta **Access Control List (ACL)**. Essa elenca tutti gli utenti e i permessi che ognuno di essi ha per ogni specifica risorsa. Eventualmente potrebbe contenere una entry aggiuntiva per regolare l'accesso di tutti quegli utenti che non hanno una specifica entry all'interno della lista (vedi world). In un sistema di questo tipo risulta semplice ottenere la lista di permessi per una certa risorsa, ma è difficile ottenere la lista di permessi per un certo utente.
- Per righe: ogni riga viene chiamata **Capability List**. Essa specifica le autorizzazioni per oggetti e operazioni di ogni specifico utente. Eventualmente un utente potrebbe essere in grado di "prestare" i permessi ad altri. Poiché queste strutture dati sono sparse in memoria, è necessario garantirne l'integrità, per esempio firmandole con un apposito token. In un sistema di questo tipo risulta semplice ottenere la lista di permessi per un certo utente, ma è più complesso ottenere la lista di permessi per una certa risorsa.

4.4 Example: UNIX File Access Control – Esempio: Access Control in UNIX

In UNIX, i file sono gestiti dal SO tramite un descrittore detto inode (index node). Esso è una struttura di controllo che contiene le informazioni che descrivono ogni specifico file e funge anche da descrittore in RAM al momento dell'apertura del file. Diversi nomi di file possono essere associati allo stesso inode, ogni inode si riferisce ad un solo file e ogni file ha un soli inode.

Le directory sono strutturate secondo un albero gerarchico: ognuna di esse può contenere file e/o altre cartelle. In memoria una directory viene rappresentata come un file che contiene la lista di file e relativi inode che contiene. Anche le directory vengono rappresentate tramite inode.

4.4.1 Traditional UNIX File Access Control

Ad ogni utente UNIX è assegnato uno specifico user ID numerico (0 per root), un gruppo primario (che generalmente contiene solo lui e ha il suo stesso nome) ed eventualmente altri gruppi di cui fa parte, identificati tramite un group ID numerico.

Ogni file viene creato da un certo utente, che diventa il suo owner. È inoltre associato al gruppo primario dell'utente oppure, se la directory ha il bit SetGID impostato a 1, al gruppo owner della directory. All'interno dell'inode si trovano 12 bit che regolano i permessi di ogni file:

- Primi 9 bit: replicano 3 volte la tripletta **R-W-X** (read, write ed execute), regolando i permessi di owner, gruppo di appartenenza e resto del mondo. I permessi di lettura e scrittura applicati a una directory rendono l'utente capace di listare i file in una directory, crearli, cancellarli e rinominarli. È importante ricordare che i permessi assegnati ad una directory possono differire da quelli assegnati ad ogni file al suo interno.
- 3 bit rimanenti: vengono utilizzati per specificare comportamenti addizionali.
 - **SetUID**: set user ID, può essere assegnato solo a un file eseguibile.
 - Assegnato a file eseguibile: quando ogni utente esegue i file, lo fa con i permessi dell'owner. In questo modo un utente senza particolari permessi è in grado di eseguire un programma che richiede privilegi maggiori di quelli che lui ha. Il cambiamento è effettivo solo per la durata dell'esecuzione de file.
 - Assegnato a directory: ignorato.

- **SetGID:** set group ID, può essere associato a un file eseguibile o a una directory.
 - Assegnato a file eseguibile: quando ogni utente esegue i file, lo fa con i permessi del gruppo proprietario.
 - Assegnato a una directory: quando un utente crea un file all'interno della directory, esso appartiene al gruppo proprietario della directory.
- **Sticky bit:** può essere associato a un file o a una directory.
 - Assegnato a un file: indica che il file deve essere mantenuto in memoria dopo l'esecuzione. Non più usato.
 - Assegnato a una directory: indica che solo l'owner di ogni file può rinominare, muovere o cancellare quel file. Viene utilizzato per gestire le cartelle a cui accedono più utenti (Es. temp).

L'utente amministratore, detto *superuser*, è identificato da un apposito user ID e ha accesso all'intero sistema. Per questo ogni file da lui creato con proprietà SetUID potenzialmente permette accesso senza controllo a qualunque utente sia in grado di eseguirlo.

Questo tipo di controllo degli accessi è adeguato quanto ci sono pochi utenti e gruppi e quando la loro gestione è sufficientemente semplice. Sono in seguito stati introdotti sistemi per rendere più potente il meccanismo: le ACL.

4.4.2 Access Control List in UNIX

È possibile associare ad ogni file una ACL in grado di definire eccezioni rispetto alla classica gestione dei file in UNIX. Ogni entry contiene lo user ID o il group ID a cui si riferisce e 3 bit che specificano i permessi nella maniera tradizionale (R-W-X). In questo caso i bit che specificano i permessi del gruppo nel sistema classico prendono il nome di maschera e indicano i permessi massimi che un utente o gruppo indicato nell'ACL può possedere (eventuali permessi sovrabbondanti verranno ignorati).

Quando un utente vuole accedere a un file con ACL vengono eseguiti 2 passaggi:

- Si cerca l'entry della ACL più piccola che contiene l'utente (inizialmente si controlla se è il proprietario, poi si controlla se c'è la entry per l'utente corrente, per il suo gruppo). Solo una entry può determinare i permessi dell'utente. Nel caso l'utente appartenga a più gruppi elencati in varie entry dell'ACL, ne viene scelta una che soddisfa il permesso necessario (se non è presente, l'accesso è negato).
- Si controlla che la entry garantisca sufficienti permessi per compiere tale operazione.

4.5 Role-Based Access Control

RBAC è basato sul ruolo che ogni utente può assumere piuttosto che sull'identità dell'utente stesso. Per ogni ruolo, che può essere correlato alla funzione che l'utente ha nell'azienda, vengono definiti i diritti di accesso necessari. Ad ogni utente possono poi essere assegnati diversi ruoli, staticamente o dinamicamente, a seconda dei loro compiti. Generalmente si cerca di rispettare il principio di least privileges, che questo sistema permette di rispecchiare fedelmente: ogni ruolo dovrebbe contenere il minimo numero di permessi necessari a svolgere tale funzione. L'utente dovrà quindi cambiarlo a seconda della funzione che dovrà svolgere.

La relazione tra utenti e ruoli è molti a molti: ogni utente può avere più ruoli e ogni ruolo può essere impersonato da più utenti. Generalmente il ruolo di ogni utente è un dato piuttosto dinamico (può variare spesso), mentre l'insieme di ruoli e i relativi permessi tendono a rimanere costanti nel tempo.

Un sistema di controllo degli accessi role-based può essere rappresentato tramite 2 matrici:

- Matrice degli utenti: mette in relazione utenti e ruoli, valorizzando una cella se l'utente può assumere tale ruolo. Tipicamente gli utenti sono molto più numerosi dei ruoli. Ogni utente può assumere multipli ruoli e ogni ruolo può essere assunto da più utenti.
- Matrice dei ruoli: mette in relazione ruoli e risorse, valorizzando adeguatamente una cella con i permessi che ha l'utente sulla risorsa. Tipicamente le risorse sono molte più dei ruoli.

4.5.1 RBAC Reference Models – Modelli di Riferimento per RBAC

Di seguito saranno descritti i modelli di riferimento per RBAC. Ogni modello contiene quelli di livello inferiore, estendendone le caratteristiche. In particolare RBAC₀ contiene le caratteristiche fondamentali, RBAC₁ le estende con la gerarchizzazione dei ruoli, RBAC₂ una serie di vincoli tra ruoli.

4.5.1.1 Base Model – RBAC₀ – Modello base

Descrive le caratteristiche fondamentali di un sistema RBAC, in particolare:

- **Utente (User):** individuo che ha accesso al sistema, identificato tramite uno user ID.
- **Ruolo (Role):** funzione o compito che devono essere svolti sul sistema. Tipicamente sono accompagnati dalla descrizione dell'autorità e delle responsabilità designate al suo svolgimento.
- **Permesso (Permission):** modalità di accesso garantita a un certo oggetto.
- **Sessione (Session):** mapping tra utenti e insieme di ruoli attivi in un certo momento.

È da notare che la relazione è molti a molti sia tra utenti e ruoli (ogni utente ha più ruoli, ogni ruolo è svolto da più utenti) che tra ruoli e permessi (ogni ruolo descrive più permessi e ogni permesso è contenuto in più ruoli). Ciò permette di garantire una granularità e una flessibilità molto maggiori rispetto a qualunque sistema DAC: un utente potrebbe infatti necessitare di accessi diversi sulla stessa risorsa in momenti diversi, ma con DAC l'unica possibilità è assegnargli i permessi maggiori necessari. L'utente dovrebbe utilizzare, per ogni sessione, il minimo insieme di ruoli che gli permettono di svolgere tale compito (least privilege).

4.5.1.2 Role Hierarchises – RBAC₁ – Gerarchie

Il secondo modello permette di creare gerarchie di utenti, rispecchiando più fedelmente la struttura gerarchica di un'organizzazione. In questo modo si possono creare ruoli di alto livello che hanno più diritti di accesso e ruoli di più basso livello che ne contengono solo un sottoinsieme. Ogni ruolo eredita implicitamente tutti i diritti dei suoi subordinati.

4.5.1.3 Constraint – RBAC₂ – Vincoli

Questo modello inserisce una serie di vincoli che possono essere inseriti tra i vari ruoli, mettendoli in relazione tra loro. In particolare:

- **Mutua esclusione:** un utente non può avere entrambi i ruoli staticamente (non possono essergli assegnati) o dinamicamente (non può averli attivi entrambi nella stessa sessione). I ruoli mutuamente esclusivi devono avere permessi non sovrapposti.
- **Cardinalità:** numero massimo di ruoli che può avere un utente staticamente (assegnati) o dinamicamente (attivi).
- **Prerequisito:** un utente non può assumere un certo ruolo se prima non ne ha assunto un altro (Es. non può assumere un ruolo di alto livello se prima non ha assunto il ruolo figlio).

5 Database and Cloud Security

5.4 Database Access Control

Gli attacchi di SQL Injection (SQLi) sono tra i più comuni e dannosi attacchi eseguiti in rete. Essi sfruttano la natura dinamica delle moderne pagine web: esse richiedono l'inserimento di informazioni all'utente memorizzandole in database e, sulla base delle informazioni fornite, sono in grado di mostrare pagine personalizzate per una migliore esperienza d'uso. Gli attacchi SQLi sfruttano ciò inviando comandi SQL malevoli al database server con lo scopo di estrarre informazioni, di modificare o cancellare dati, di eseguire comandi del sistema operativo oppure di lanciare attacchi DoS.

5.4.1 A Typical SQLi Attack – Un tipico attacco SQL Injection

Un attacco SQLi sfrutta una vulnerabilità di sicurezza della parte di applicazione che va ad interagire con il database (es. che si occupa di generare le query) per fare in modo che venga eseguito codice non autorizzato. I passi necessari all'esecuzione di un attacco di questo tipo sono:

1. L'attaccante trova una vulnerabilità in una pagina web dinamica e inietta un comando SQL inviandolo al server in modo che sia immerso all'interno di dati affidabili e quindi accettati da un eventuale firewall.
2. Il web server riceve il codice malevolo e lo invia al web application server.
3. Il web application server riceve il codice e lo invia al database server.
4. Il database server esegue il codice e restituisce i dati.
5. Il web application server genera la pagina dinamica includendo i dati estratti, che l'attaccante può quindi leggere.

5.4.2 The Injection Technique – Tecnica per l'iniezione di codice

La tecnica più comune è quella di terminare prematuramente una query e appendere ad essa il comando desiderato. La parte successiva può eventualmente essere commentata (--) in modo che venga ignorata. Il codice ricevuto dal database avrà una forma analoga a questa: `SELECT * FROM OrdersTable WHERE ShipCity = "Boston"; DROP table OrdersTable --" AND Customer = "John The Ripper"`. Le tre parti evidenziate sono rispettivamente l'inizio della query originale, la parte iniettata (contenente una prima parte per fare "funzionare" la query originale, una parte malevola e il simbolo del commento) e la parte finale della query originale commentata.

5.4.3 SQLi Attack Avenues and Types – Modalità di attacco SQLi

Esistono diversi tipi di attacco SQLi che differiscono per come il codice malevolo viene inserito all'interno della query:

- **User input:** l'attaccante invia il codice inserendolo adeguatamente all'interno dell'input di una form web. Le web application sono in grado di accedere al contenuto dei campi, inviati tramite GET o POST HTTP, come se fosse una variabile.
- **Server variables:** il codice viene iniettato all'interno di variabili contenute nell'header HTTP, negli header di altri protocolli di rete oppure nelle variabili d'ambiente che, se esse vengono utilizzate senza controllo (la procedura di controllo e correzione è detta *sanitization*) potrebbero generare una vulnerabilità. Questo tipo di attacco è particolarmente facile utilizzando la tecnica degli header HTTP, in quanto possono essere generati in maniera semplice.
- **Second-order injection:** sfrutta l'incompletezza dei meccanismi di controllo dell'input per eseguire codice malevolo già presente all'interno del server o del database. L'input che modifica la query proviene direttamente dal server, non dall'attaccante. Per esempio un campo viene valorizzato con la stringa `"; DROP TABLE x --"` e viene richiamata una query che utilizza quel campo per la selezione.
- **Cookies:** le pagine web spesso utilizzano cookies per mantenere informazioni relative al client. Un attaccante può alterare tali cookies inserendo codice malevolo.
- **Physical user input:** l'injection avviene utilizzando un input costruito appositamente per eseguire l'attacco. I sistemi che presentano vulnerabilità per questo attacco sono quelli che prevedono lettura di codici a barre, tag RFID, ...

Gli attacchi possono inoltre essere classificati a seconda del canale di comunicazione usato:

- **Inband:** il canale di comunicazione usato è lo stesso per iniettare il codice e ottenere il risultato. Tra essi abbiamo:
 - **Tautology:** l'attaccante fa in modo che una condizione sia sempre vera. Per esempio `SELECT [...] WHERE name = "" OR 1 = 1 -- AND [...]`. In questo modo si può, per esempio, disabilitare il controllo delle password.
 - **End-of-line comment:** la parte successiva al codice iniettato viene modificata introducendo commenti, quindi non eseguito.
 - **Piggybacked query:** l'attaccante aggiunge query a quella legittima. Sfrutta la possibilità di eseguire più query nella stessa stringa di codice.
- **Inferential:** non c'è scambio di dati, ma l'attaccante è in grado di ricostruire informazioni osservando il risultato o il comportamento del server sottomettendo richieste particolari. Tra essi troviamo:
 - **Illegal/logically incorrect query:** l'attaccante ottiene informazioni sulla struttura sottostante del database sottomettendo query deliberatamente errate e analizzando il tipo di errore restituito. Spesso infatti la pagina di errore contiene più informazioni del necessario.
 - **Blind SQL injection:** l'attaccante sfrutta differenze di comportamento nella pagina web quando vengono sottomesse query di un certo tipo.
- **Out-of-band attack:** i dati sono ottenuti sfruttando un canale differente da quello tramite cui è stata inviata la richiesta, per esempio una email.

5.4.4 SQLi Countermeasures

Gli attacchi SQLi sono molto variabili e diffusi, quindi non è sufficiente una sola contromisura: è necessario un insieme di tecniche integrate. Possono essere divise in 3 macro-categorie:

- **Defensive coding:** la maggior parte degli attacchi riescono perché lo sviluppatore ha scritto codice insicuro. Le tecniche più comuni di defensive coding sono:

- **Manual defensive coding:** lo sviluppatore si impegna a scrivere codice che validi l'input prima di sottmetterlo al database. Comprendono il controllo dei tipi dei dati inseriti e il pattern matching.
- **Parametrized query:** viene specificata in maniera accurata la struttura della query SQL, permettendo l'inserimento di un certo insieme di parametri separatamente, facendo sì che non possa esistere input in grado di modificare la struttura della query stessa.
- **SQL DOM:** classi che permettono di controllare automaticamente tipi di dati ed escaping delle query, incapsulando la query in un'apposita struttura detta DOM.
- **Detection:** metodi che permettono di rilevare la volontà di eseguire un attacco SQLi.
 - **Signature based:** le query vengono confrontate con un database di pattern costantemente aggiornato per verificare che siano "legali".
 - **Anomaly based:** si tenta di definire il pattern tipico della query e si fa in modo di scoprire se la query non lo rispetta. È necessaria inizialmente una fase di training, per "insegnare" il pattern al sistema.
 - **Code analysis:** utilizza apposite suite che si occupano di rilevare potenziali vulnerabilità nel codice.
- **Run-time prevention:** tecniche che controllano a tempo di esecuzione se la query è conforme o meno al modello che il programma si aspetta. Esistono svariati tool off-the-shelf.

5.8 Cloud Computing

Oggi molte grandi aziende stanno spostando parte delle loro funzioni connesse all'informatica su infrastrutture distribuite online dette **cloud**.

Il NIST definisce **cloud computing** un modello che permette l'accesso a un pool di risorse computazionali configurabili in maniera conveniente, on demand, tramite rete e da qualunque parte del mondo. Tali risorse sono in grado di adattarsi, come quantità e caratteristiche, alle necessità dell'utente.

Più in dettaglio le caratteristiche di un sistema di cloud sono:

- **Accesso tramite rete:** le risorse sono accessibili attraverso la rete, utilizzando meccanismi standard, tramite client (thin o thick) o altri servizi.
- **Elasticità:** il pool di risorse è in grado di espandersi o ridursi a seconda delle necessità del cliente "istante per istante".
- **Misurazione:** è possibile controllare e ottimizzare l'uso delle risorse in maniera automatica oltre che in maniera esplicita e trasparente da parte dell'utente.
- **On-demand self-service:** il cliente può unilateralmente ottenere risorse computazionali senza necessità di richiedere l'intervento umano nel momento in cui gli servono.
- **Resource pooling:** le risorse fornite sono utilizzate per soddisfare i bisogno di più clienti, utilizzando un sistema detto *multi-tenant* in cui le risorse fisiche e virtuali possono essere riassegnate in base alle necessità. Generalmente il cliente non ha la possibilità di controllare o addirittura conoscere l'esatta locazione delle risorse, ma potrebbe essere in grado di specificare la locazione a un più alto livello di astrazione (es. stato, datacenter, ...).

Il NIST divide il cloud computing in 3 livelli innestati:

- **Software as a Service (SaaS):** il cloud provider fornisce all'utente software applicativo che viene eseguito su risorse appartenenti all'infrastruttura del cloud provider ed è accessibile tramite la rete generalmente attraverso interface web. Risparmia al cliente la necessità di installare, mantenere e aggiornare il software.
- **Platform as a Service (PaaS):** il cloud provider fornisce all'utente una piattaforma su cui il cliente può eseguire o sviluppare software. Può mettere a disposizione diversi blocchi, tra cui un insieme di tool di sviluppo per diversi linguaggi di programmazione, runtime environment, middleware, database, ... Permette al cliente di non doversi preoccupare dell'acquisto e della gestione delle risorse computazionali, pur potendo utilizzare software specificamente prodotto.
- **Infrastructure as a Service (IaaS):** il cloud provider fornisce all'utente l'accesso alla struttura più a basso livello del cloud, fornendo macchine virtuali con sistemi operativi ed hardware astratto. Le risorse comprendono potenza di calcolo, spazio di memoria, reti, e altre risorse che consentono al cliente di sviluppare ed eseguire software arbitrario, senza però essere vincolati all'hardware fisico.

A seconda della modalità di condivisione (o di non condivisione) esistono 4 **deployment models**:

- **Cloud pubblico:** l'infrastruttura è messa a disposizione del pubblico o di un grande gruppo di industrie ed è posseduta da un'organizzazione che vende il servizio. Il cloud provider è responsabile sia dell'infrastruttura che del controllo di dati e operazioni.
- **Cloud privato:** l'infrastruttura messa a disposizione è esclusiva di una certa organizzazione, ma può essere gestita da una terza parte. Il cloud provider è responsabile dell'infrastruttura, ma non del controllo.
- **Cloud comunitario (community):** l'infrastruttura è condivisa da diverse organizzazioni con bisogni simili. Potrebbe essere gestito dalle organizzazioni stesse o da una terza parte.
- **Cloud ibrido:** l'infrastruttura è la composizione di o più tipi di cloud appena descritti, che consentono portabilità pur rimanendo entità separate.

5.8.1 Cloud Computing Reference Architecture – Architettura di riferimento per il Cloud

La definizione del NIST non descrive implementazione e soluzioni da adottare per la realizzazione di un sistema di cloud, ma semplicemente cosa dovrebbe fornire un servizio di cloud. In questo modo illustra e permette di comprendere i diversi tipi di cloud, di compararne le caratteristiche e facilitare l'analisi di standard di sicurezza, interoperabilità, portabilità e implementazione.

I 5 principali attori in un sistema di cloud sono:

- **Cloud consumer:** colui che mantiene le relazioni e utilizza i servizi messi a disposizione dal cloud provider.
- **Cloud provider:** responsabile della messa a disposizione del servizio agli interessati. Eventualmente potrebbe inglobare anche altri dei ruoli descritti in seguito.
- **Cloud auditor:** terza parte indipendente in grado di valutare servizi, performance e sicurezza dell'implementazione del cloud.
- **Cloud broker:** colui che gestisce l'uso, le performance e consente la fruizione dei servizi negoziando tra il provider e il consumer. Entra in gioco quando i servizi cloud sono troppo complessi per essere gestiti direttamente dal cliente, supportandolo in 2 modi:
 - **Service aggregation:** combina multipli servizi per soddisfare le necessità del cliente oppure per minimizzare i costi o ottimizzare le performance.
 - **Service arbitrage:** aggrega diversi servizi, ma a differenza dell'aggregazione è in grado di prelevarli da differenti provider.
- **Cloud carrier:** intermediario che permette connettività e tra provider e consumer. Comprende tutte le infrastrutture di rete, connettività e trasporto, garantendo un certo servizio (SLA, Service Level Agreement) al provider in modo che egli possa a sua volta fornire un certo servizio al cliente.

5.9 Cloud Security Risks and Countermeasures – Rischi e Contromisure per servizi Cloud

I rischi di un Sistema Cloud sono generalmente gli stessi di un qualunque altro Sistema IT, a cui si aggiungono tuttavia problemi specifici legati al fatto che l'azienda perde parte del suo controllo su risorse, servizi e applicazioni dovendo comunque garantire sicurezza e privacy. Tra essi abbiamo:

- **Abuso e uso non permesso del cloud computing:** in quanto relativamente semplici da ottenere, le risorse vengono utilizzate per condurre vari tipi di attacco, tra cui spamming e DoS. Le contromisure possibili sono rendere più severe le procedure di registrazione e valutazione, analisi del traffico dei clienti, monitoraggio di blacklist pubbliche per impedire la registrazione di clienti malevoli.
- **Interfacce ed API non sicure:** il client provider mette a disposizione una serie di API, che tuttavia potrebbero non essere sicure sotto diversi punti di vista (autenticazione, controllo degli accessi, crittografia, monitoring delle attività). Le contromisure da prendere sono l'analisi del modello di sicurezza delle interfacce messe a disposizione e l'utilizzo di procedure di autenticazione, controllo degli accessi e crittografia avanzate.
- **Malicious insiders:** i dipendenti del cloud provider hanno a disposizione l'hardware fisico del sistema e hanno il controllo su aspetti di sicurezza estremamente critici. Ciò è ovviamente fondamentale per poter svolgere il loro compito, ma potrebbe generare rischi non previsti. Le contromisure possibili sono specificare i requisiti delle risorse umane direttamente nel contratto, richiedere trasparenza nella gestione delle informazioni e della sicurezza, stabilire un sistema di notifica per eventuali problemi di sicurezza.

- **Problemi legati alla condivisione:** per garantire scalabilità dei servizi, spesso le risorse messe a disposizione sono eseguite su sistemi virtualizzate su una stessa CPU. Tali sistemi potrebbero non essere disegnati per offrire proprietà di isolamento così robuste da rendere un sistema multi-tenant sufficientemente sicuro. Alcune contromisure sono installare e configurare il sistema per renderlo il più sicuro possibile, monitorare continuamente l'ambiente per rilevare attività o cambiamenti non autorizzati, promuovere un'autenticazione robusta e un controllo degli accessi ben organizzato, stabilire nel contratto l'obbligo di applicare patch per la risoluzione di vulnerabilità, eseguire operazioni di auditing.
- **Perdita o diffusione non autorizzata dei dati:** una breccia di sicurezza che causa perdita o diffusione non autorizzata dei dati potrebbe essere notevolmente grave. Ci si può proteggere rinforzando il controllo degli accessi, garantendo l'integrità dei dati in transito e memorizzati con la crittografia, utilizzando politiche sicure per la generazione, memorizzazione, gestione e distruzione delle chiavi.
- **Account o service hijacking:** è possibile che un'entità non autorizzata si impadronisca dell'account o del servizio, per esempio rubando le credenziali. In questo modo avrebbe accesso a tutte le aree critiche, mettendo a rischi integrità, confidenzialità e disponibilità delle risorse. Le contromisure comprendono il divieto di condivisione delle credenziali, l'utilizzo di autenticazione a più fattori, pratiche di intrusion detection.
- **Unknown risk profile:** comprende tutti i rischi legati al fatto che non si conosce l'architettura e altri dettagli interni dell'infrastruttura di cloud, così come alcune politiche di sicurezza. Le contromisure sono la pubblicazione di logs, dati, di caratteristiche dell'infrastruttura (patch level, firewall, ...) e il monitoraggio.

5.10 Data Protection in the Cloud – Protezione dei dati nel Cloud

I dati memorizzati nel cloud possono essere compromesse in vari modi (es. cancellazione, modifica, estrazione dal contesto, cancellazione delle chiavi di accesso, ...). Tale rischio è più grave in quanto le risorse del cloud sono condivise. Esistono due meccanismi principali di condivisione:

- **Modello multi-instance:** ad ogni cliente viene assegnato un DBMS in esecuzione su una macchina virtuale a cui lui solo ha l'accesso. In questo modo può gestire autonomamente il controllo degli accessi, le autorizzazione e gli altri dettagli delle politiche di sicurezza.
- **Modello multi-tenant:** esiste un unico database condiviso, le cui parti vengono messe a disposizione dei vari clienti permettendone o meno l'accesso. È il cloud provider che si deve occupare di mantenere il database sicuro.

È da ricordare che i dati devono essere mantenuti sicuri sia mentre sono memorizzati che mentre sono in transito e che l'accesso ad essi deve essere controllato. Si possono sfruttare tecniche avanzate di controllo degli accessi e crittografia. Essi permettono di impedire la lettura dei dati da parte di chiunque, compreso del cloud provider fino a che le chiavi vengono mantenute sicure (se anche accedesse all'hardware, i dati sarebbero crittografati), ma non proteggono da corruzione e DoS.

5.11 Cloud Security as a Service – SecaaS

Con SecaaS si intende una serie di servizi di sicurezza messi a disposizione dal provider che scaricano la responsabilità della gestione della sicurezza dalle spalle dell'azienda. Possono comprendere servizi di autenticazione, anti-virus, antimalware/spyware, di intrusion detection e di gestione di eventi di sicurezza.

Le categorie principali come definite dalla Cloud Security Alliance sono:

- **Identity and access management (IAM):** servizi che consentono di gestire l'accesso alle risorse assicurando che l'identità di una certa entità sia verificata in maniera affidabile e garantendo il corretto livello di accesso a seconda della sua identità (autenticazione e controllo degli accessi).
- **Data loss prevention (DSP):** monitoraggio, protezione e verifica della sicurezza dei dati durante la memorizzazione, il trasporto e l'uso.
- **Web security:** protezione real-time offerta tramite l'installazione di apposite apparati oppure utilizzando parte dell'infrastruttura cloud come proxy che si occupa di aggiungere uno strato di protezione che può comprendere antivirus, antimalware, ...
- **E-mail security:** esegue controlli di sicurezza sulle mail entranti e uscenti, proteggendo da phishing, allegati malevoli, rinforzando le politiche di sicurezza dell'azienda, bloccando lo spam, ...
- **Security assessment:** esegue audit su altri servizi cloud.

- **Intrusion management:** comprende meccanismi per l'intrusion detection, prevention e response da applicare ai punti di ingresso del cloud e dei server all'interno del cloud.
- **Security information and event management (SIEM):** aggrega log e dati provenienti da reti reali e virtuali, applicazioni e sistemi, correlandole ed analizzandole per fornire report e alert in tempo reale, eventualmente richiedendo l'intervento umano.
- **Encryption:** si occupa di crittografia per dati memorizzati, cloud, email, trasmissioni in rete e altre informazioni.
- **Business continuity and disaster recovery:** misure e meccanismi che consentono di garantire la ripresa del servizio in caso di interruzioni. Comprendono sistemi di backup con meccanismi di failover e disaster recovery. Devono essere realizzati su infrastrutture flessibili, ridondanti, monitorate e geograficamente distribuite.
- **Network security:** comprende servizi di sicurezza che si occupano di allocazione, accesso, distribuzione, monitoraggio e protezione delle risorse sottostanti. Tra di essi troviamo firewall e sistemi di protezione da DoS.

9 Firewalls and Intrusion Prevention Systems – Firewall e Intrusion Prevention

I firewall sono un mezzo per proteggere i sistemi locali o le reti da tutte quelle minacce di sicurezza che provengono dal mondo esterno, quindi da Internet, ma che allo stesso tempo permettono l'accesso delle risorse in entrambe le direzioni.

9.1 The Need For Firewalls – Lo scopo dei Firewall

La gestione dei sistemi informatici all'interno delle aziende e organizzazioni è avvenuta in questo modo:

- Dati e capacità di processarli concentrate in un unico mainframe centrale a cui i terminali sono direttamente connessi.
- Reti locali (LAN) che connettono i terminali tra loro e al mainframe.
- Reti che interconnettono più LAN, collegando tra loro PC, server e mainframe.
- Reti aziendali, che connettono LAN multiple e geograficamente distribuite formando una WAN.
- Connettività Internet, a cui tutte le reti locali sono collegate.

Le informazioni e i servizi messi a disposizione da Internet sono fondamentali per l'organizzazione e permettono, fra le altre cose di fare interagire con il resto del mondo le risorse locali. Ciò però introduce minacce di sicurezza. Il gran numero di dispositivi connessi e la loro diversità, tuttavia, rende impossibile configurare singolarmente ognuno di essi: è necessario un sistema scalabile e facilmente gestibile. Vengono quindi inseriti dei firewall tra le reti locali e Internet, facendo in modo che controllino le informazioni che passano da e verso Internet, proteggendo le reti interne dagli attacchi esterni e generando un singolo punto di controllo per l'audit. I firewall permettono quindi di realizzare la defense in depth, aggiungendo uno strato di isolamento tra interno e esterno.

9.2 Firewall Characteristics and Access Policy – Caratteristiche dei Firewall

Di seguito verranno elencate le specifiche di un firewall:

- Tutto il traffico entrante e uscente passa attraverso di esso. Ogni altro accesso alla rete deve essere impedito.
- Solo il traffico autorizzato da specifiche policy deve poter passare attraverso il firewall.
- Il firewall non è vulnerabile in quanto utilizza sistemi operativi sicuri.

La parte più critica nella progettazione e implementazione di un buon firewall è la costruzione delle policy di accesso, ovvero la specifica di quale traffico è autorizzato a passare o meno a seconda di indirizzo, protocollo, applicazione e contenuto. Di seguito una lista di caratteristiche che possono essere usate come filtri per un firewall:

- **Indirizzo IP e informazioni dei protocolli:** indirizzo IP destinazione e sorgente, numeri di porta, direzione del traffico (inbound/outbound) e tutte le altre informazioni relative ai protocolli di trasporto utilizzati. Vengono generalmente utilizzati per limitare l'accesso a specifici servizi.
- **Protocollo di livello applicazione:** è possibile limitare i protocolli applicativi consentiti o meno. Generalmente lo si fa utilizzando un application-level gateway che fa relay della connessione.
- **Identità dell'utente:** il traffico viene autorizzato o meno a seconda dell'utente che lo ha generato. È necessario avere un meccanismo di autenticazione sicura, come IPSec.

- **Attività:** basa il filtro su considerazioni riguardanti le modalità di utilizzo della rete, come tempo o frequenza delle richieste.

Un firewall deve avere le seguenti capacità:

- Rappresenta un singolo punto di controllo (tutte le funzionalità sono concentrate su un singolo sistema o un insieme molto ridotto) che si occupa di mantenere gli utenti non autorizzati al di fuori della rete protetta, impedisce l'utilizzo di servizi vulnerabili da o verso la rete interna e permette protezione contro IP spoofing.
- Rappresenta un punto in cui si possono controllare e monitorare tutti gli eventi relativi alla sicurezza. Vi si può fare auditing oppure utilizzarlo come trigger per allarmi di sicurezza.
- Può essere utilizzato per altri compiti non direttamente legati alla sicurezza, come per esempio il NAT oppure applicazioni che si occupano della gestione dei log dell'utilizzo di Internet.
- Può essere utilizzato come piattaforma per IPSec, quindi per la creazione di VPN.

A causa della sua natura ha però alcune limitazioni:

- Non può proteggere da qualunque attacco che bypassa il firewall, per esempio perché passa per una connessione parallela (per esempio la connessione dati sui telefoni).
- Non può proteggere completamente da attacchi insider, per esempio eseguiti da dipendenti in collaborazione con altri attaccanti esterni.
- Non può proteggere da dispositivi infetti portati in azienda e connessi alla rete interna.

9.3 Types of Firewalls – Tipi di Firewall

Un firewall può monitorare il traffico su diversi livelli della pila protocollare, a seconda delle policy che sono necessarie. Può inoltre essere impostato come filtro positivo (default deny/discard, tutto viene bloccato tranne specifiche comunicazioni autorizzate) oppure come filtro negativo (default permit/forward, tutto viene fatto passare tranne specifiche comunicazioni che vengono scartate). Di seguito una descrizione dei principali tipi di firewall.

9.3.1 Packet Filtering Firewall

Analizza i pacchetti IP (livello di rete) entranti e uscenti sulla base delle seguenti informazioni:

- **Indirizzo IP sorgente**
- **Indirizzo IP destinatario**
- **Indirizzi di trasporto sorgente e destinatari** (nel caso di TCP o UDP il numero di porta)
- **Campo "Protocol" del pacchetto IP** (definisce il tipo di protocollo di trasporto usato)
- **Interfaccia** (porta fisica su cui viene ricevuto o su cui deve essere inviato il pacchetto)

A seconda delle policy stabilite decide o meno se inoltrare il pacchetto. Se non si riesce a far corrispondere nessuna policy ad un certo pacchetto si possono adottare due tecniche:

- **Default = discard:** tutto ciò che non è espressamente permesso è proibito. Migliore dal punto di vista della sicurezza ma più complessa da configurare. Impone generalmente molte limitazioni agli utenti.
- **Default = forward:** tutto ciò che non è espressamente proibito è permesso. Migliora la facilità di utilizzo del sistema, ma fornisce una sicurezza minore. L'amministratore dovrebbe creare regole apposite nel caso in cui vengano rilevate nuove vulnerabilità.

Un firewall di questo tipo ha sicuramente il vantaggio della semplicità, della trasparenza e della velocità. Ha però una serie di debolezze più o meno gravi:

- Non può prevenire attacchi specifici per alcune applicazioni in quanto non analizza dati del protocollo applicativo.
- A causa delle poche informazioni che può vedere di ogni pacchetto, il logging è scarsamente utile.
- Generalmente non supportano schemi di user authentication, in quanto lavorano a livelli bassi della pila protocollare.
- Sono vulnerabili ad attacchi che sfruttano vulnerabilità dei protocolli TCP/IP (es. spoofing).
- A causa del numero limitato di variabili utilizzate per il controllo degli accessi sono suscettibili ad attacchi dovuti ad implementazioni errate.

Alcuni degli attacchi che possono essere condotti ad un packet filtering firewall sono:

- **IP spoofing:** l'attaccante trasmette pacchetti dall'esterno inserendo nei pacchetti inviati un indirizzo IP interno all'azienda. Il firewall lascerà passare il pacchetto credendo che provenga da un utente autorizzato. Una contromisura è scartare tutti i pacchetti con indirizzo IP interno che provengono da interfacce esterne.
- **Source routing:** l'attaccante specifica la route che il pacchetto dovrà percorrere attraverso Internet nel tentativo di bypassare le misure di sicurezza. Una contromisura è scartare tutti i pacchetti che hanno attiva questa opzione.
- **Tiny fragment:** la frammentazione IP potrebbe impedire al firewall di analizzare efficacemente un pacchetto in quanto le informazioni sarebbero contenute in molti pacchetti diversi (solo il primo della sequenza viene analizzato). Una contromisura è specificare la quantità minima di header che deve essere contenuta nel primo pacchetto, pena la sua eliminazione (se il primo pacchetto viene scartato, tutti i successivi frammenti verranno a loro volta scartati).

9.3.2 Stateful Inspection Firewalls

Il problema più grave dei packet filtering firewall è che analizzano ogni pacchetto indipendentemente, senza considerare informazioni di più alto livello come il contesto. Per esempio un packet filtering firewall deve permettere pacchetti entranti su qualunque porta alta (>1000) per permettere la ricezione di pacchetti di una connessione TCP uscente. La conoscenza del contesto del pacchetto permette di creare politiche più efficienti e sicure, per esempio autorizzare pacchetti verso le porte alte solo se provengono da una delle connessioni correntemente stabilite, memorizzate in un'apposita tabella.

Di questo si occupa lo stateful packet inspection firewall. Esso ripete i controlli fatti dal packet filtering firewall, ma memorizza anche informazioni relative alle connessioni TCP aperte (tra cui, per esempio, il numero di sequenza per evitare il session hijacking) rendendo più precisi ed efficaci i controlli.

9.3.3 Application-Level Gateway

Agisce facendo relay di tutto il traffico a livello di applicazione. L'utente contatta il gateway utilizzando un'applicazione TCP appositamente modificata (eventualmente si può limitare il tipo di applicazioni utilizzabili), specificando anche il destinatario remoto della connessione e autenticandosi al gateway. Solo dopo l'autenticazione il gateway si occuperà di inoltrare i pacchetti provenienti dall'utente all'host remoto e viceversa.

Un firewall di questo tipo risulta più efficiente del semplice filtraggio di pacchetti e permette di generare log più completi e di eseguire audit del traffico di rete. Introduce tuttavia overhead dovuto al tempo di processamento dei pacchetti (devono essere stabilite 2 connessioni invece che una sola e il gateway deve analizzare i pacchetti che passano attraverso di lui).

9.3.4 Circuit-Level Gateway

Spezzano in due la connessione TCP, non permettendo connessioni end-to-end tra utente e destinatario remoto. Il gateway tuttavia non esamina il contenuto dei pacchetti, ma inoltra semplicemente i pacchetti relativi a connessioni permesse.

Tipicamente viene utilizzato per connessioni dall'interno all'esterno quando l'amministratore si fida dei dipendenti, mentre per connessioni provenienti dall'esterno si utilizza un Application-Level Gateway.

Un'implementazione molto utilizzata è SOCKS. In essa un utente che vuole comunicare con l'esterno si connette al servizio SOCKS sulla porta TCP 1080, si autentica con uno dei metodi permessi e richiede il servizio di relay. Il server SOCKS valuta la richiesta e, in caso positivo, stabilisce la connessione TCP tra sé stesso e l'host remoto. Nel caso di applicazioni che utilizzano UDP, i pacchetti UDP vengono inviati fin tanto che la connessione TCP tra utente e server è mantenuta in piedi.

9.4 Firewall Basing – Posizionamento dei Firewall sugli Host

Generalmente i firewall sono implementati su macchine dedicate che eseguono sistemi operativi comuni, UNIX o Linux oppure direttamente su router o switch di rete a cui vengono aggiunti appositi moduli software.

9.4.1 Bastion Host

Sistema identificato dall'amministratore del firewall come punto critico della sicurezza della rete. Tipicamente viene utilizzato come piattaforma per un gateway application-level o circuit-level. Esso generalmente ha le seguenti caratteristiche:

- Esegue un sistema operativo sicuro (sistema hardened).
- Sono installati solamente quei servizi considerati essenziali (es. proxy DNS, FTP, HTTP, SMTP).
- Potrebbe richiedere autenticazione prima di permettere agli utenti di accedere ai proxy. Ogni proxy potrebbe avere il suo specifico metodo di autenticazione.
- Ogni proxy è configurato in modo da supportare solamente i comandi necessari.
- Ogni proxy è configurato in modo da permettere l'accesso solo da specifici host.
- Ogni proxy mantiene informazioni di log e audit, tra cui il numero e la durata delle connessioni effettuate.
- Ogni proxy è un modulo software molto piccolo, disegnato specificamente per servizi di sicurezza. Grazie alla sua semplicità è possibile riconoscere velocemente vulnerabilità.
- Ogni proxy è indipendente dagli altri proxy installati sullo stesso bastion host. In questo modo può essere disattivato, sostituito o aggiornato senza problemi per gli altri servizi che l'host offre.
- Ogni proxy non accede al disco se non per leggere la configurazione iniziale ed essere eseguito. Le porzioni di file system contenenti tali dati devono essere configurate in sola lettura, per rendere più complessa l'installazione di cavalli di Troia o altri file malevoli.
- Ogni proxy è eseguito come utente non privilegiato e in una directory sicura.

9.4.2 Host-Based Firewalls

Moduli software utilizzati per rendere più sicuri i singoli host. Tipicamente vengono installati sui server, permettendo i seguenti vantaggi rispetto a un firewall generico tradizionale:

- Le regole di filtering possono essere cucite addosso al singolo host. Diversi host hanno diverse necessità.
- La protezione è garantita indipendentemente dalla topologia. Sia attacchi interni che esterni devono passare attraverso il firewall.
- Utilizzato insieme a firewall generici fornisce un ulteriore strato di protezione.

9.4.3 Personal Firewall

Firewall implementati direttamente all'interno di un PC o una workstation. Tipicamente sono più semplici di quelli installati sui server o di quelli stand alone.

Hanno come scopo quello di impedire l'accesso non autorizzato al computer (generalmente default deny in download se non in servizi specifici, come file sharing, FTP, ...) e possono allo stesso tempo monitorare il traffico verso l'esterno rilevando la presenza di worm o malware (generalmente default permit in upload).

9.5 Firewall Location and Configuration – Posizionamento dei Firewall in rete e Configurazione

Un firewall viene posizionato in modo da fungere da barriera protettiva tra rete esterna (untrusted) e interna (trusted). È importante quindi capire come posizionare i firewall e come configurarli.

9.5.1 DMZ Networks – Demilitarized Networks

Configurazione molto comune in passato che include un tratto di rete aggiuntiva compresa tra i firewall interno, posizionato all'interno della rete aziendale, ed esterno, posizionato ai confini della rete aziendale. La zona compresa tra essi prende il nome di DMZ e viene utilizzata per collegare sistemi che devono essere accessibili dall'esterno ma che richiedono comunque un certo grado di controllo.

Il firewall esterno provvede meccanismi di controllo degli accessi e una protezione basica per la rete interna (spesso è implementato direttamente sul router), mentre quello interno aggiunge ulteriori filtri per proteggere i server interni dalla rete esterna e la DMZ da attacchi provenienti dall'interno. Eventualmente la rete interna potrebbe essere divisa in più compartimenti separati.

9.5.2 Virtual Private Network

Insieme di computer interconnessi tramite rete insicura (es. Internet) che fanno uso di servizi di crittografia e appositi protocolli per garantire sicurezza (impedire eavesdropping e entry-point per utenti non autorizzati), comportandosi

sostanzialmente come se fossero all'interno di una rete privata. Una VPN è infatti più economica che il posizionamento di linee private. La crittografia può essere applicata dal software dei firewall o dai router.

Il protocollo più utilizzato per la creazione di VPN è IPSec. La base per IPSec (ovvero il modulo software che si occupa di applicare la crittografia) dovrebbe essere posizionato il più esternamente alla rete possibile: in questo modo i pacchetti possono arrivare al firewall già decrittati e quindi analizzabili. Possibili candidati sono il router di confine o il firewall stesso (più sicuro del router in quanto teoricamente impenetrabile).

9.5.3 Distributed Firewalls – Firewall distribuiti

Una configurazione distribuita comprende firewall stand-alone e firewall host-based che lavorano cooperando sotto il controllo di un unico centro di amministrazione. Esistono appositi tool che permettono all'amministratore di configurare firewall personali e stand-alone distribuiti all'interno dell'intera rete, permettendo protezione da attacchi interni in maniera specifica per ogni dispositivo grazie ai firewall host-based e protezione da attacchi esterni grazie ai firewall stand-alone (interni e esterni).

9.5.4 Summary of Firewall Locations and Topology – Riassunto sulle tecniche di posizionamento dei Firewall

- **Host-resident firewall:** firewall software personali e per server, utilizzati da soli oppure facenti parte di un più complesso Sistema e fornendo uno strato di protezione addizionale.
- **Screening router:** router posizionato al confine della rete interna che si comporta come packet filtering firewall. Adatto per piccoli uffici (SOHO, small office/home office).
- **Single bastion inline:** un singolo firewall è posizionato tra i router interni ed esterni, implementando filtri stateful o proxy applicativi. Adatto a piccole e medie imprese.
- **Single bastion T:** esiste una terza rete separata collegata al firewall su una porta separata, la DMZ, dove si trovano i server accessibili dall'esterno. Adatto a imprese medio-grandi.
- **Double bastion inline:** la DMZ viene posizionata tra i due bastion firewall. Adatta a imprese grandi o organizzazioni governative.
- **Double bastion T:** la DMZ è posizionata in una rete separata, collegata ad un'apposita interfaccia del bastion firewall. Adatta a grandi imprese o organizzazioni governative.
- **Distributed firewall:** forma più complessa, utilizzata da imprese molto grandi o organizzazioni governative.